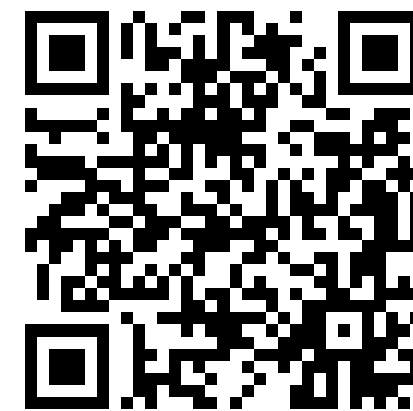


超級電腦入門教學

方育斌、張朝宜、陳威宇



教材：https://github.com/robinfang7/nchc_hpc_tutorial

Agenda

1. 國網中心的超級電腦

1. CPU叢集系統 (創進一號、台灣杉三號)
2. GPU叢集系統 (晶創25、台灣杉二號)
3. iService計算資源服務網

3/3 : 超級電腦入門教學(線上)

3/26 : 創進一號(CPU叢集)(線上)

4/8 : 晶創25主機(GPU叢集) (線上)

2. 軟體

1. Slurm任務調度
2. Lmod軟體管理
3. Singularity/Apptainer容器
4. 安裝軟體

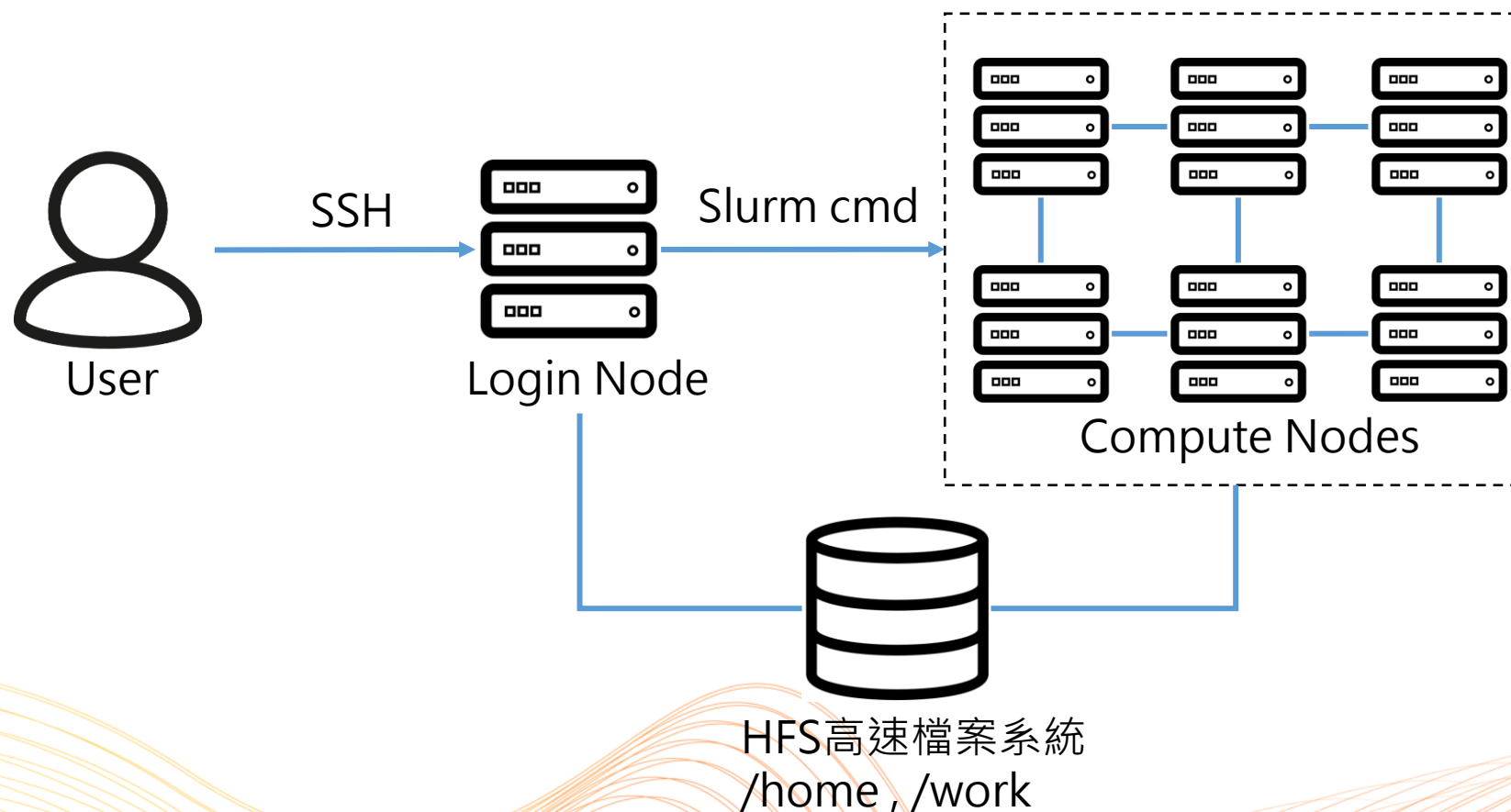
3. 實作展示

1. 登入主機、查詢資源、submit job
2. CPU叢集案例
3. GPU叢集案例

4. 問題與討論



超級電腦 / 高效能計算



創進一號Forerunner 1

- 創進一號 (Forerunner I) 為國網中心最新建置的高速計算主機，具備558台計算伺服器，共可提供62,496個計算核心；每台計算伺服器配備2顆56核心Intel Xeon Platinum 8480+ 2.0GHz處理器、InfiniBand 200Gb/s高速網路、512GB記憶體。實際整體最佳效能約為3.5PFlops。
- 創進一號 (Forerunner I) 為泛用型計算主機，可廣泛提供各領域如物理、化學、數學、大氣科學、工程應用、生命科學等研究所需，適合產官學研各用戶使用，更鼓勵有大計算需求之研究多加運用。除專業套裝軟體外，系統亦提供數種編譯器與MPI函式庫供研究人員自行開發計算程式。



台灣杉三號 (TAIWANIA 3)

- 台灣杉三號(Taiwania 3)為國內提供開放服務申請的最大CPU高速計算主機(2021年)，擁有900個計算節點。台灣杉三號全系統共計 50,400 個計算核心(Cores)、與台灣杉二號共用平行檔案系統總容量約為 9.4PB，搭配InfiniBand HDR100 100Gbps高速網路，整體效能可達2.7千兆次浮點運算(PFLOPS)。
- 本系統為泛用型計算主機，可提供多種領域的計算服務，包含物理、化學、數學、大氣科學、工程應用、生命科學等。除商業套裝軟體外，系統亦提供數種編譯器與本MPI函式庫供研究人員自行開發計算程式之用。



創進一號、台灣杉三號規格比較

	創進一號	台灣杉三號
總算力	3.5 PFLOPS	2.7 PFLOPS
節點數	558	900
檔案系統總容量	9.3 PB	9.4 PB
節點CPU	2顆Intel Xeon Platinum 8480+ 2.0GHz	2顆 Intel® Xeon® Platinum 8280 2.4GHz
節點CPU核心	112	56
節點記憶體容量	512GB	192GB
網路頻寬	InfiniBand NDR200 200Gb/s	InfiniBand HDR100 100Gbps
費率	0.1/0.2/0.2/0.37	0.08/0.08/0.16/0.16

四種CPU小時費率為國科會計畫、學術計畫、政府與法人計畫、企業與個人計畫

系統架構與計算資源

創進一號共有 552台x86計算節點與6台x86繪圖節點 (總計1116個處理器、62,496個核心、12個3D繪圖用A40 GPU) 與40台ARM計算節點 (總計80個處理器、5,760個核心)。

- 558 台x86計算節點由雙 CPU 插槽組成，每個插槽具為 Intel Xeon Platinum 8480+, 56cores, 2.0GHz。
- 40台ARM計算節點由雙 CPU 插槽組成，每個插槽為NVIDIA Grace CPU Superchip 使用Arm Neoverse V2核心，內建72核心，2.6GHz。

節點種類	總個數 (節點)	單位計算資源 (節點)				
		CPU 插槽數	CPU核心數	記憶體 (GB)	Nvidia A40	InfiniBand NDR 200Gb/s
X86	552	2	112	512	-	1
X86繪圖節點	6	2	112	512	2	1
ARM	40	2	144	240	-	1

整體效能最高達3.5 Petaflops，9.2 PB儲存容量。

晶創25 Nano5

- 晶創25(英文名稱：Nano 5)為新一代加速器叢集式超級電腦，計有21台NVIDIA H100以及16台NVIDIA H200AI伺服器，共有296片NVIDIA Hopper計算加速器，為台灣發展AI及高效能運算的關鍵基礎設施。晶創主機也承襲「晶創計畫」的願景，助力半導體產業發展AI應用，也為半導體產業其他各種應用及研發，提供強大且高速的計算環境。晶創25主機採用NVIDIA Hopper計算加速器，於2025年6月TOP500首次排行118名，算力(Rmax)達13.06PetaFLOPS，可大幅提升人工智慧模型訓練與科學模擬的運算效率。此主機適用於產業界、政府機構、學術單位以及研究機構，為各類用戶提供高效能的運算支援，助力AI全面推動。



台灣杉二號Taiwania 2 / TWCC台灣AI雲

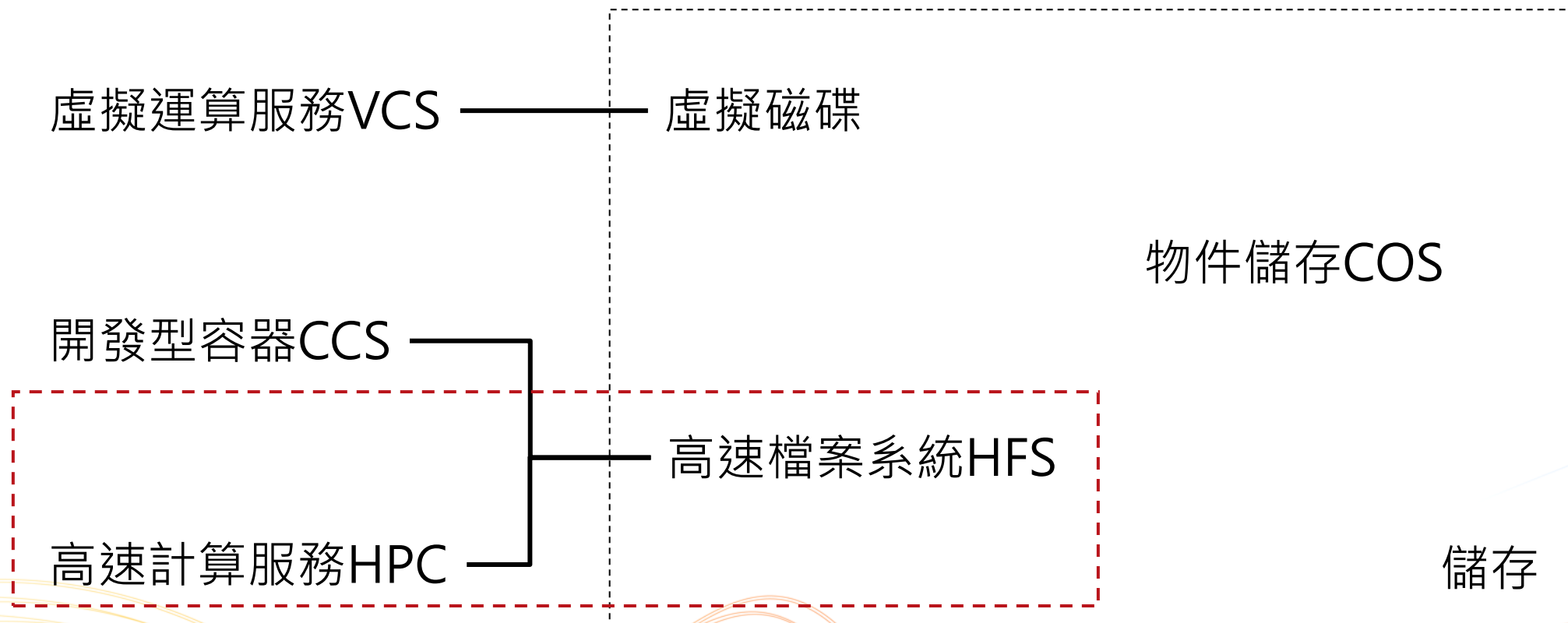
- 「台灣杉二號」(Taiwania 2)為AI超級電腦主機，共運用2,016個NVIDIA Tesla V100 32GB GPU，以9 PFLOPS (每秒執行9千兆次浮點運算)的優異效能，在2018年底公布的全球500大高速計算主機 (TOP500) 中，排名第20名，僅次於美國、中國、瑞士、日本、德國、韓國、義大利、法國；能源效率 (Green500) 排名第10名，僅次於日本、美國、西班牙、中國，雙雙締造台灣超級電腦入榜有史以來最好的成績，也寫下我國科技國力發展的重要里程碑。
- 「台灣杉二號」(Taiwania 2)超級電腦由252個節點組成，每個節點包含2顆CPU及8顆最先進GPU，其主機架構設計與國際趨勢同步。在科學家運用大數據進行深度學習時，可表現出更優質的性能；此外，在節能方面，「台灣杉二號」的能源效率達11.285 GF/W，計算量在9 PFLOPS時，用電798 KW，亦為台灣史上最節能的高速計算主機。



TWCC軟體架構



TWCC服務說明



晶創26 Nano4

- 「晶創26」為國家推動AI、高效能運算與科研自主能力的重要基礎設施，由國研院國網中心規劃建置。作為「前瞻基礎建設」與「AI十大建設」的重要一環，「晶創26」不僅象徵臺灣算力硬體的進化，更是主權AI與技術自主發展的關鍵推手。其中「Nano4」實測最佳用電功率為 2.214 百萬瓦 (MW)，最高實測效能 (Rmax) 為81.55 PFlops，為目前臺灣最快、運算密度最高的超級電腦。
- 「晶創26」採用雙重算力架構設計，包括「Nano4」(H200 架構系統)，以及最新一代的 GB200 NVL72架構節點，兼具通用科學模擬與AI訓練的高效能運算需求。

11/2025 Top500 #29



GPU主機規格

	晶創26	晶創25	台灣杉二號
建置時間	2026	2025	2018
總算力	81.55 PFLOPS	16 PFLOPS	9 PFLOPS
節點數	220台NVIDIA H200+ 2櫃NVIDIA GB200 NVL72	21台H100節點+16台H200節點	252台V100節點
檔案系統總容量	25PB	9.3 PB	10 PB
節點CPU	2 x Intel Xeon Platinum處理器/ 72 x NVIDIA Grace處理器	2顆Intel Xeon(R) Platinum 8480+(8480CL)	Intel Xeon Gold CPU x 2
節點GPU	8張NVIDIA H200 GPU/ 72xBlackwell GPU	8片NVIDIA H100 GPU 8片NVIDIA H200 GPU	Nvidia Tesla V100 w/32GB x 8
節點記憶體容量	2TB / 13.5TB	2TB	768 GB
網路頻寬	InfiniBand NDR 400 Gb/s	計算伺服器間以8個Infiniband NDR 400Gbs網路埠連接 以2個 Infiniband NDR 200Gbs網 路埠連接至儲存系統	InfiniBand EDR100 100Gbps

GPU Spec

	H100 SXM5 80 GB	H200 SXM 141 GB	Tesla V100 SXM2 32 GB
Release Date	Oct. 2022	Nov. 2024	Mar. 2018
Process Size	5nm	5nm	12nm
Transistors	80,000 million	80,000 million	21,100 million
Memory Size	80 GB	141 GB	32 GB
Memory Type	HBM3	HBM3e	HBM2
Bandwidth	3.36 TB/s	4.89 TB/s	898.0 GB/s
FP16 (half)	267.6 TFLOPS	267.6 TFLOPS	31.33 TFLOPS
FP32 (float)	66.91 TFLOPS	66.91 TFLOPS	15.67 TFLOPS
FP64 (double)	33.45 TFLOPS	33.45 TFLOPS	7.834 TFLOPS
TDP	700 W	700 W	250 W
GPU小時費率	25/50/50/120	30/60/60/150	86(企業與個人計畫)

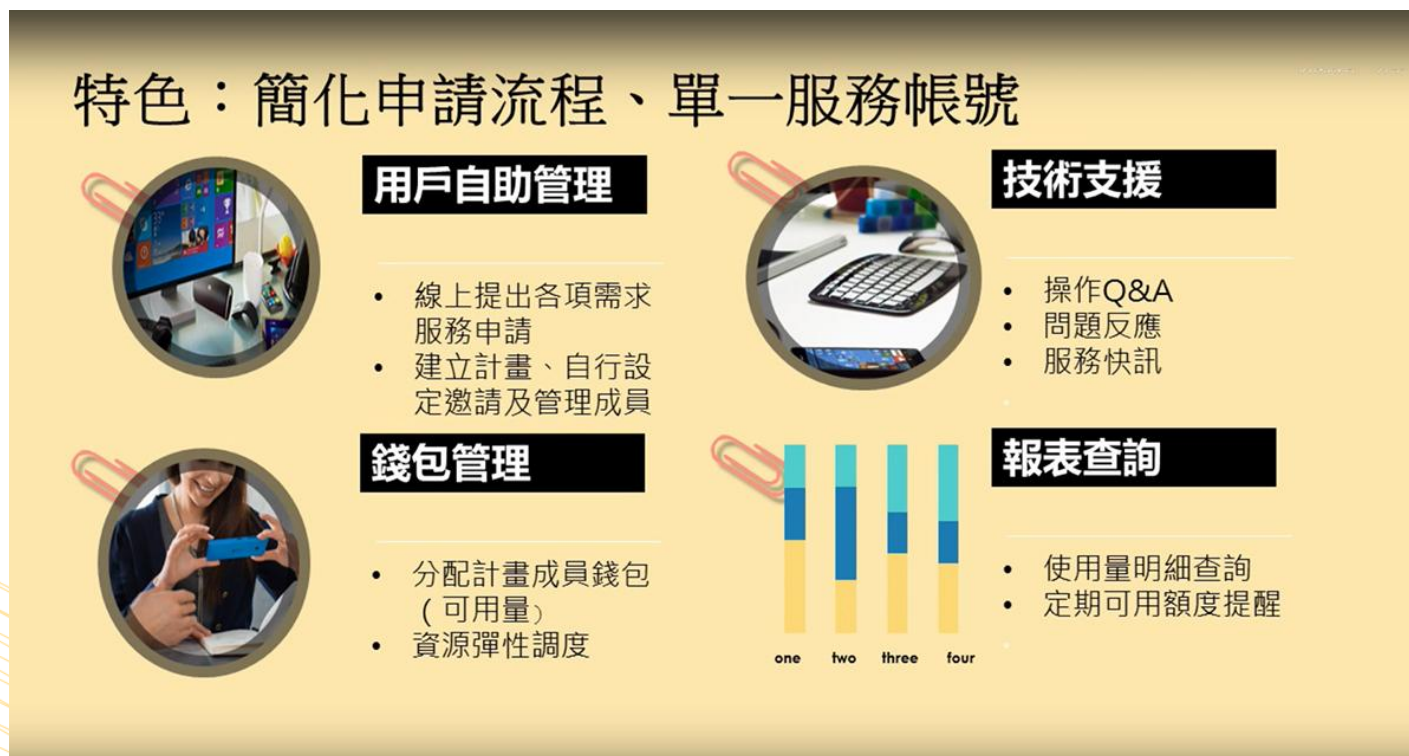
四種GPU小時費率為國科會計畫、學術計畫、政府與法人計畫、企業與個人計畫

<https://www.techpowerup.com/gpu-specs/tesla-v100-sxm2-32-gb.c3185>

<https://www.techpowerup.com/gpu-specs/h100-sxm5-80-gb.c3900>

<https://www.techpowerup.com/gpu-specs/h200-sxm-141-gb.c4255>

- <https://iservice.nchc.org.tw/>
- 提供國網中心各服務平台的帳戶申請及管理，包含計畫成員管理、預算和經費管理、使用資源報表查詢、平台操作技術支援等功能，其中計算資源申請服務，如創進一號、臺灣AI雲(TWCC)等，透過iService計算資源服務網提供服務。



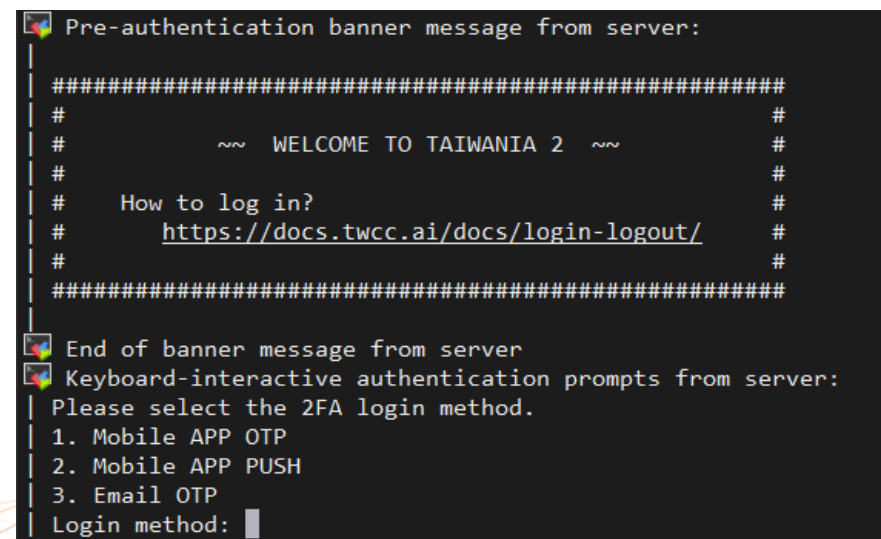
帳號有兩種

■ iService會員帳號

- 登入iService網站、TWCC網站
- 帳號的格式是電子信箱

■ 主機帳號

- 登入HPC主機，創進一號、晶創主機、Taiwania 2



主機帳號與OTP設定

修改主機帳號基本資料

主機帳號

主機帳號

██████ (啟用)

主機密碼

變更主機密碼

OTP 載具: 有效

刪除使用者載具

:::最近一次變更主機密碼日期: 2025-02-04
09:19:16

最近一次成功刪除載具日期:

注意: 已發送APP 載具註冊通知信件

使用 IDExpert 手機 APP(APP OTP 或 Push)進行登入節點雙因子(2FA) 驗證作業前, 請至 ████████ 信箱收取載具註冊通知信件, 並依內容指示完成 APP 載具註冊作業。

▶ Email OTP 電子郵件: ████████

說明:

1. 登入節點雙因子(2FA) 驗證作業方式, 選項說明:

1). Mobile APP OTP: 需安裝APP, 並完成註冊作業。開啟 APP 後取得 OTP 碼以完成 2FA 驗證。

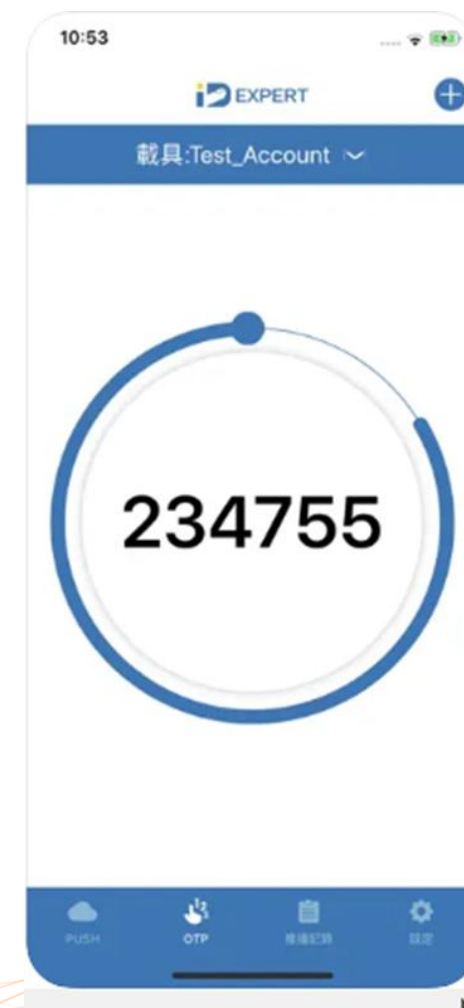
2). Mobile APP PUSH: 需安裝 APP, 並完成註冊作業。由 login node 觸發推播驗證機制至用戶的手機以完成 2FA 驗證。

3). Email OTP: 將透過您註冊於 iService E-mail 取得 OTP 驗證碼以完成 2FA 驗證。

2. 註冊與登入作業, 請參考線上說明文件。

3. 用戶更換手機需重新安裝 APP 或原安裝的 APP 刪除後重新安裝, 請參考線上說明文件第3點, 重建載具。

4. 透過 APP QRCode 進行註冊服務, 每位帳號僅可註冊一支手機。



- https://iservice.nchc.org.tw/module_page.php?module=nchc_service#nchc_service/nchc_service.php?action=nchc_motp_unix_account_edit_v3
- https://iservice.nchc.org.tw/nchc_service/nchc_service_qa_single.php?qa_code=774

高速檔案系統 HFS Hyper File System

	晶創25	台灣杉二號/台灣杉三號	創進一號
免費額度	/home 100GB /work 100GB (國科會計畫1536GB)	/home 100 GB /work 100 GB (國科會計畫1500GB)	/home 100 GB /work 100 GB (國科會計畫1500GB)
最高容量	/home 10240 GB (10TB) /work 204800 GB (200TB)	/home 10240 GB (10TB) /work 204800 GB (200TB)	/home 1024 GB (1TB) /work 204800 GB (200TB)
Hostname	nano5.nchc.org.tw	xdata1.twcc.ai	f1-ilgn01.nchc.org.tw
Port	port 2222	port 22	port 22

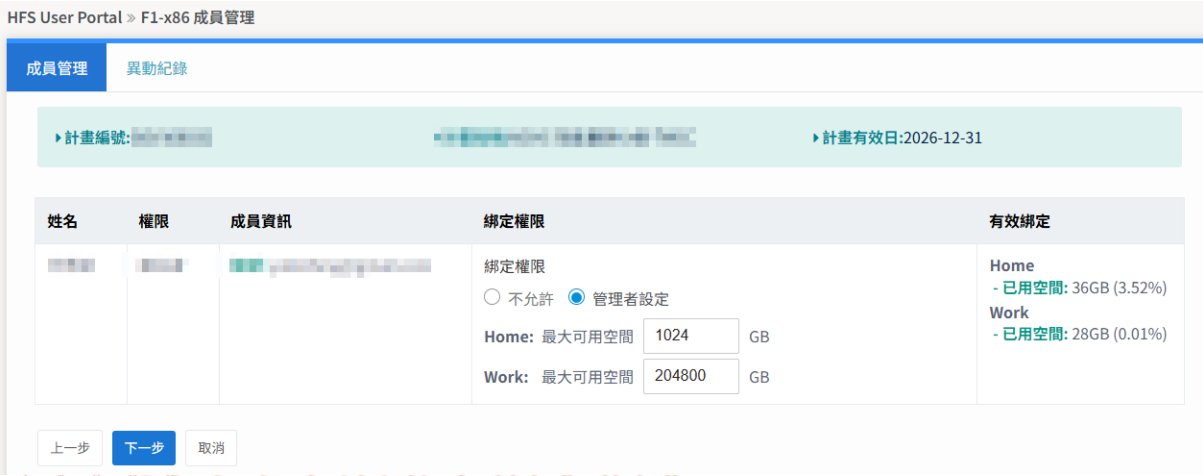
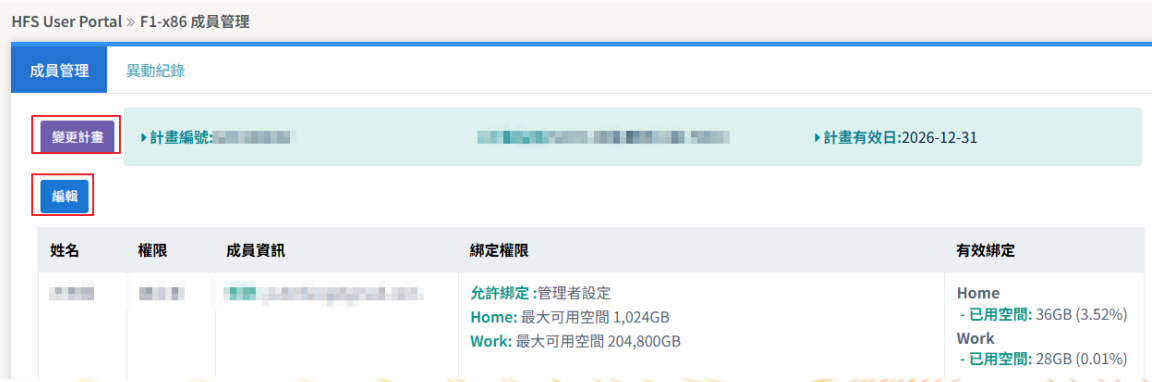
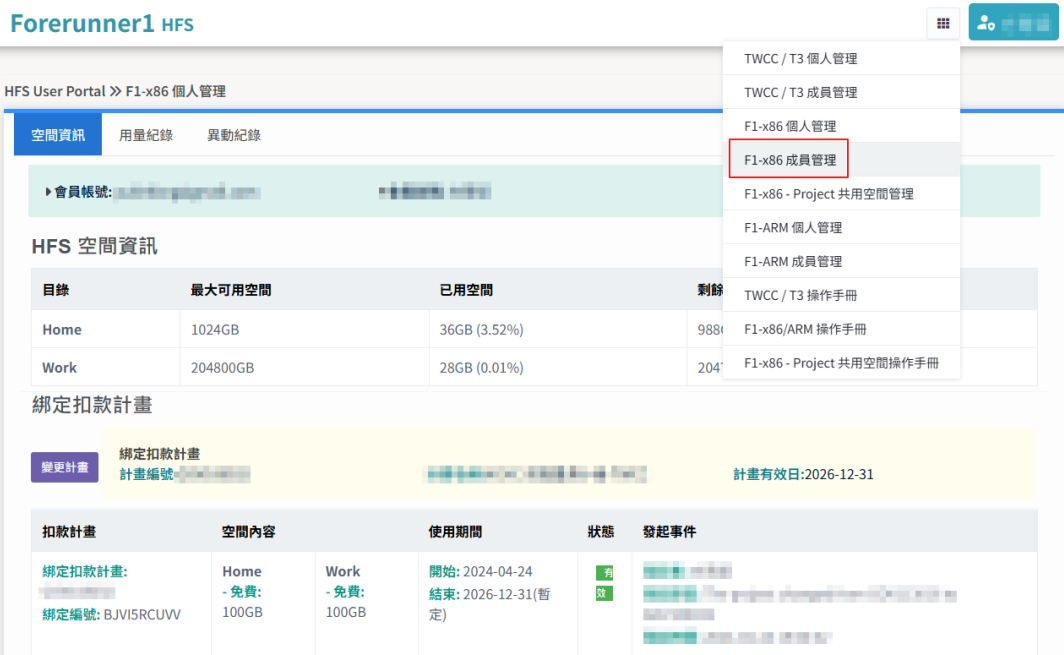
收費方法：每日計費，每天根據前一日的用量平均值進行扣款，以更實際地反應用戶的使用情況。

HFS儲存空間：(*Home、Work與Project 費率相同) (單位：元)

計畫類型	每日1 GB
國科會計畫	0.0069
學術計畫	0.035
政府與法人計畫	0.07
企業與個人計畫	0.14

設定與查詢HFS 容量

計劃建立者和管理者有權限設定HFS容量



查詢HFS容量的指令

F1

```
[user@ilgn01 ~]$ mmlsquota --block-size auto home1 work1
```

Block Limits							File Limits					Remarks
Filesystem	type	blocks	quota	limit	in_doubt	grace	files	quota	limit	in_doubt	grace	
home1	USR	25.36G	1T	1.098T	637.6M	none	145361	0	0	98	none	
work1	USR	28.37G	200T	200.1T	0	none	10	0	0	0	none	

Nano5

```
[user@cbi-lgn01 ~]$ hfsquota
```

PATH	USED	HARD LIMIT	USAGE %	STATUS	
/home/user	264,538,181,632 B	10,995,116,277,760 B	2	ACTIVE	264,538,181,632 bytes /1024/1024/1024=246GB
/work/user	11,766,642,118,656 B	219,902,325,555,200 B	5	ACTIVE	

T2

```
[user@un-ln01 ~]$ /usr/lpp/mmfs/bin/mmlsquota --block-size auto fs01 fs02
```

Block Limits							File Limits					Remarks
Filesystem	type	blocks	quota	limit	in_doubt	grace	files	quota	limit	in_doubt	grace	
/work fs01	USR	14.52T	20T	20.1T	0	none	2672451	0	0	0	none	NCHC_AIcls.twcc.ai
/home fs02	USR	186.1G	10T	10.1T	640M	none	100269	0	0	40	none	NCHC_AIcls.twcc.ai

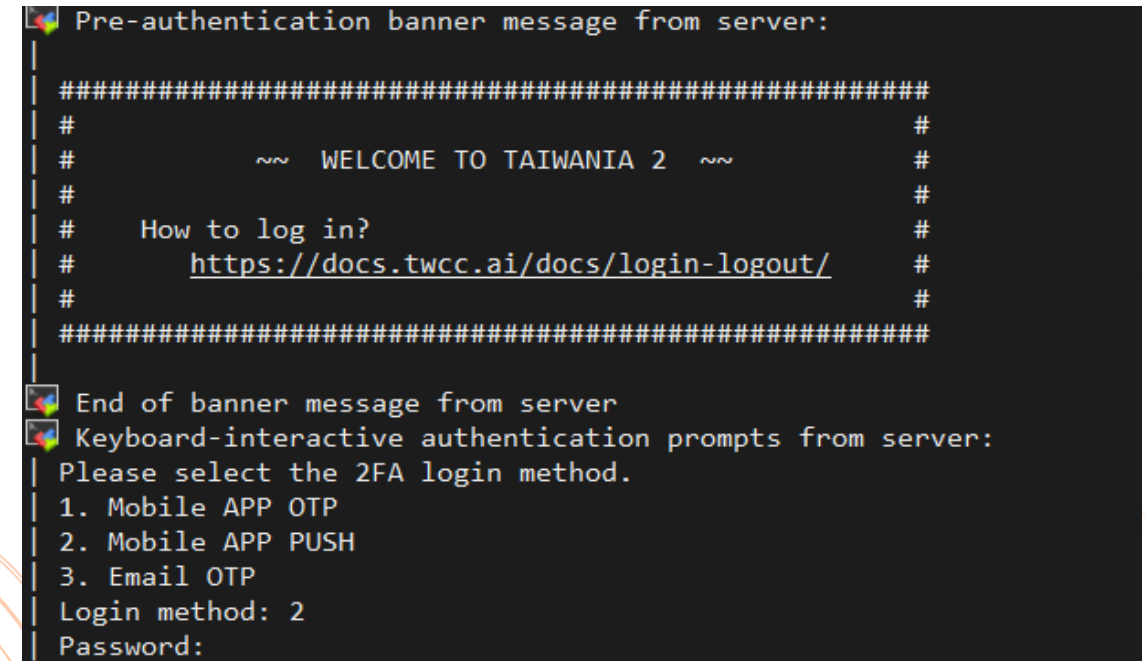
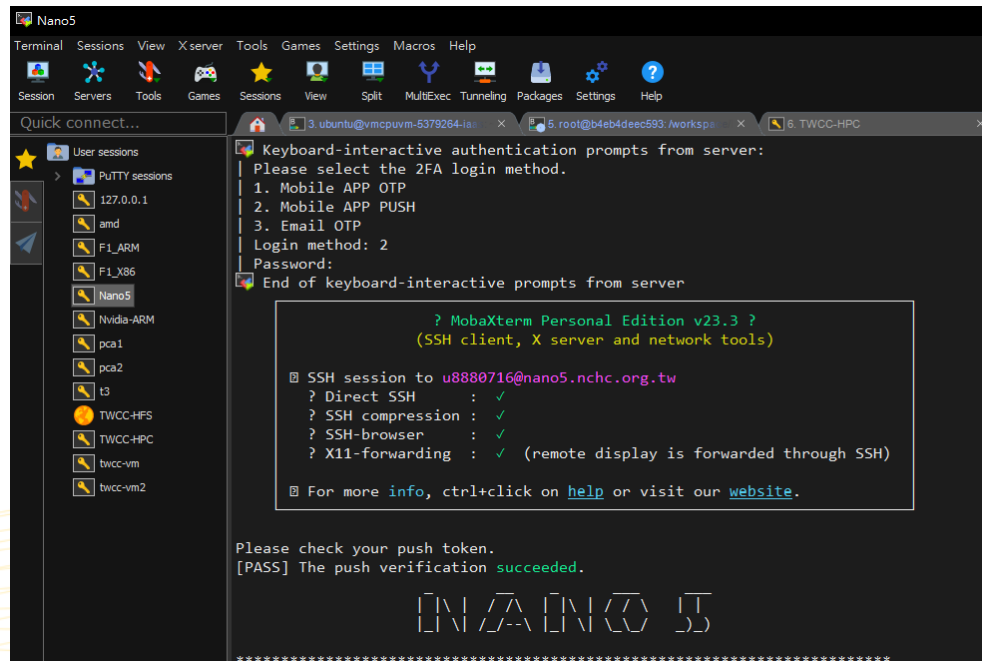
登入節點

■ SSH Tools

- ▣ MobaXterm, Putty for Windows
- ▣ Terminal for Mac and Linux

■ 雙因子(2FA) 驗證

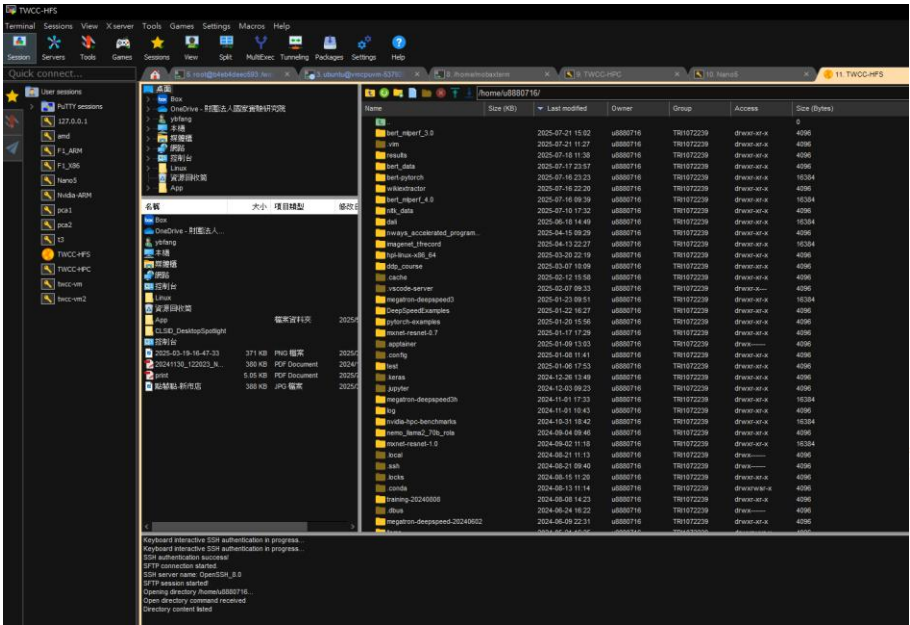
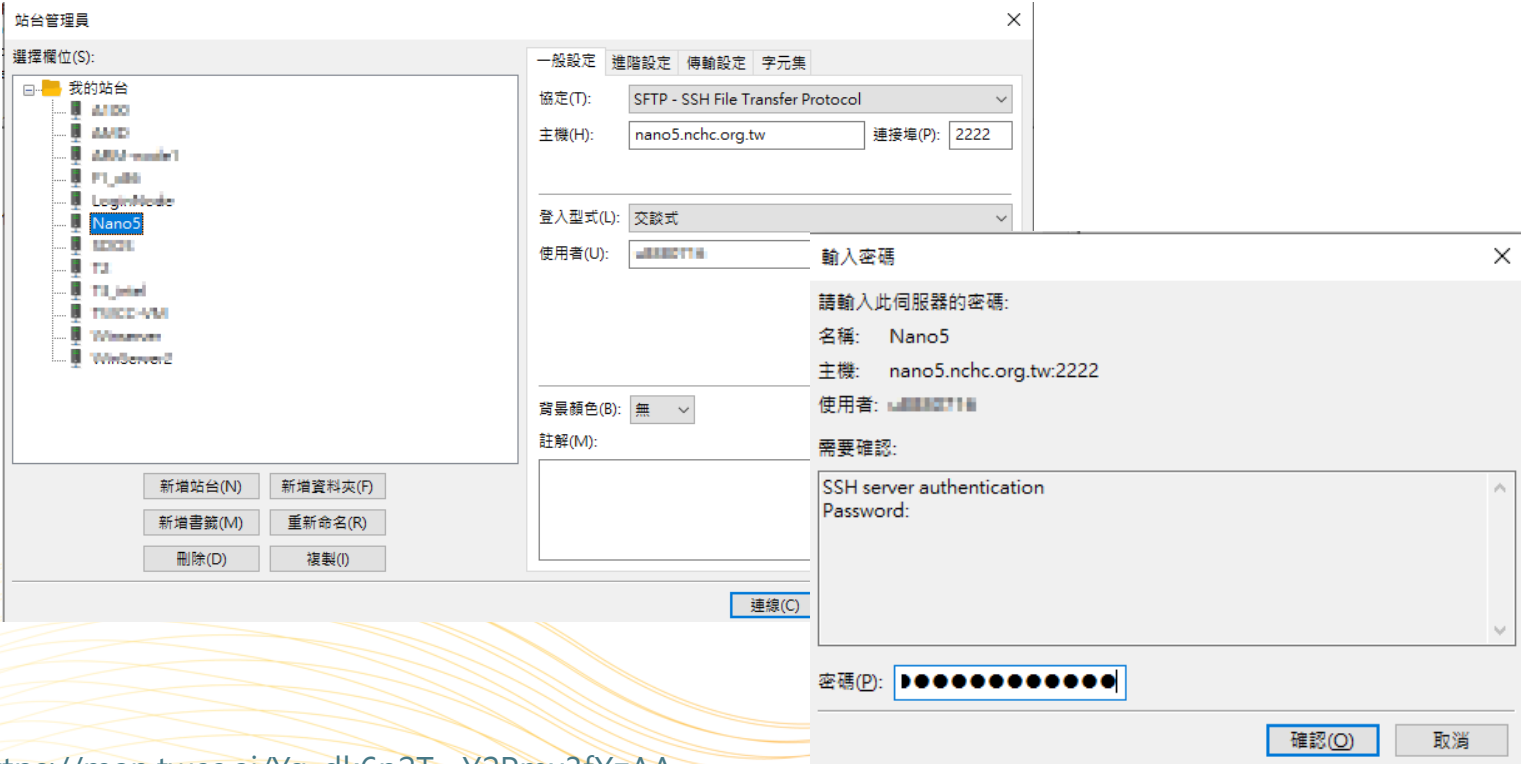
	Host name	Port
創進一號 x86	f1-ilgn01.nchc.org.tw	22
創進一號 arm	f1-nlgn01.nchc.org.tw	22
晶創主機	nano5.nchc.org.tw	22
台灣杉二號	ln01.twcc.ai	22



資料傳輸節點

- SFTP Tools
 - Filezilla, WinSCP, MobaXterm
 - sftp command
- 雙因子(2FA) 驗證

	Host name	Port
創進一號 x86	f1-dtn-vip.nchc.org.tw	22
創進一號 arm	f1-nlgn01.nchc.org.tw	22
晶創主機	nano5.nchc.org.tw	2222
台灣杉二號	xdata1.twcc.ai	22



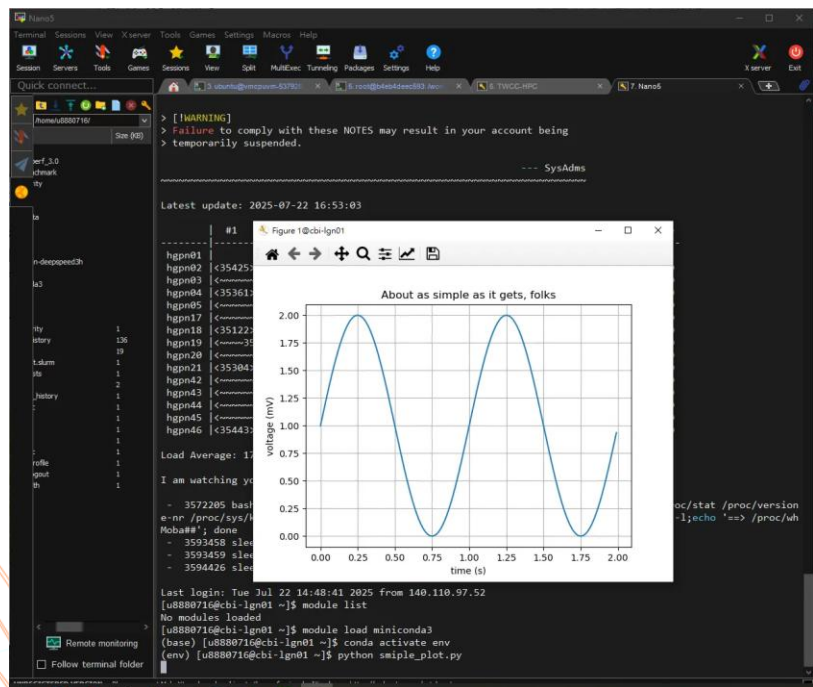
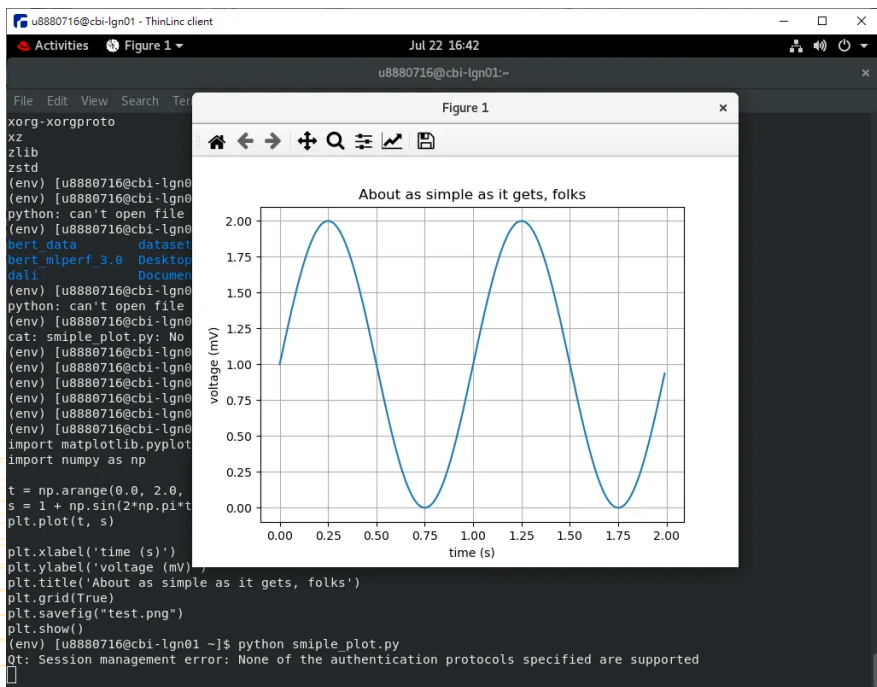
登入晶創主機的互動式繪圖節點

■ Tools

- ThinLinc for Windows, Mac, and Linux
- MobaXterm for Windows

■ 雙因子(2FA) 驗證

	Host name	Port
創進一號	f1-intgpn01.nchc.org.tw	1111
晶創主機	nano5.nchc.org.tw	22



HPC Cluster Software

- Slurm任務調度工具，取得、查詢計算資源、提交工作。
- Lmod環境管理工具，引用已安裝的編譯器、函式庫、應用程式。
- Singularity/Apptainer容器，引用已封裝依賴項與應用程式的獨立環境。
- 安裝軟體
 - 在HPC系統 (無sudo權限)
 - 在有sudo權限的環境(客製化容器打包)

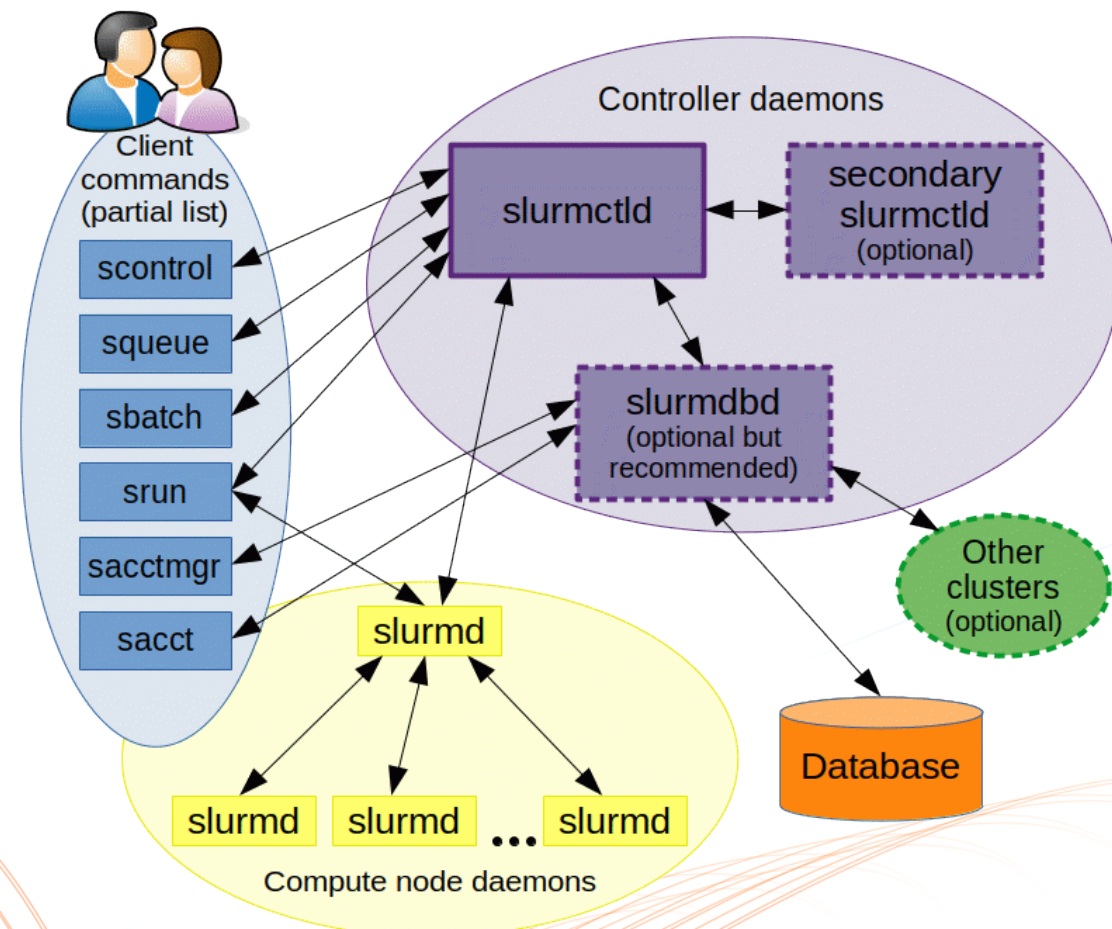
Slurm（極簡Linux資源管理工具，英文：Simple Linux Utility for Resource Management，取首字母，簡寫為SLURM），是一個用於 Linux 和 Unix 內核系統的自由開源的任務調度工具，Top500超級電腦廣泛採用。

Slurm的功能。

- 資源分配
Slurm 可以根據用戶提交的作業需求（例如節點數、CPU 核數、記憶體大小等）動態分配叢集中的計算資源，支援獨佔或非獨佔模式，保證資源得到最有效的利用。
- 作業排程與調度
Slurm 根據優先順序、作業依賴性以及當前資源狀態，將待執行作業排入隊列並分配至合適的計算節點上。它支援多種排程策略（如先進先出、優先權調度、搶占式調度等），使得系統在面對大量作業請求時能保持高效率。
- 作業管理與監控
用戶可以使用一系列命令（如 `sbatch` 提交批次作業、`srun` 執行互動作業、`squeue` 查詢作業狀態、`scancel` 取消作業等）來管理和監控作業進度，並透過記帳功能追蹤各作業的資源使用情況。

Slurm architecture

- **slurmctld daemon**：是 Slurm 的中央管理進程，負責監控集群資源和工作負載。
 - 資源監控: 追蹤集群中所有節點的可用資源，例如 CPU、記憶體等。
 - 作業排程與分配: 接收用戶提交的作業請求，根據資源需求和排程策略，將作業分配到可用的計算節點上。
 - 高可用性: 可以配置備份控制器 (backup slurmctld) 以應對主控制器故障，確保系統的穩定運行。
- **slurmd daemon**：在集群中的每個計算節點上運行
 - 節點管理: 負責管理計算節點，監控節點的狀態，並向 slurmctld 回報。
 - 作業執行: 接收來自 slurmctld 的作業指令，在本地節點上啟動、執行和監控作業。
 - 狀態回報: 將節點和作業的狀態資訊回傳給 slurmctld。
- **slurmdb daemon**：Slurm 使用資料庫來儲存和管理集群的配置資訊、作業歷史記錄和帳戶資訊等。這有助於進行資源管理和追蹤。



Slurm entities

Node

- 叢集中的伺服器。這些是實際執行使用者作業的硬體資源。

Partition

- 分區將叢集中的節點按照用途、硬體配置或管理政策劃分為不同群組，類似於工作隊列。
- 用戶在提交作業時需要指定目標分區，每個分區可以設定不同的資源限制與排程政策。

Job

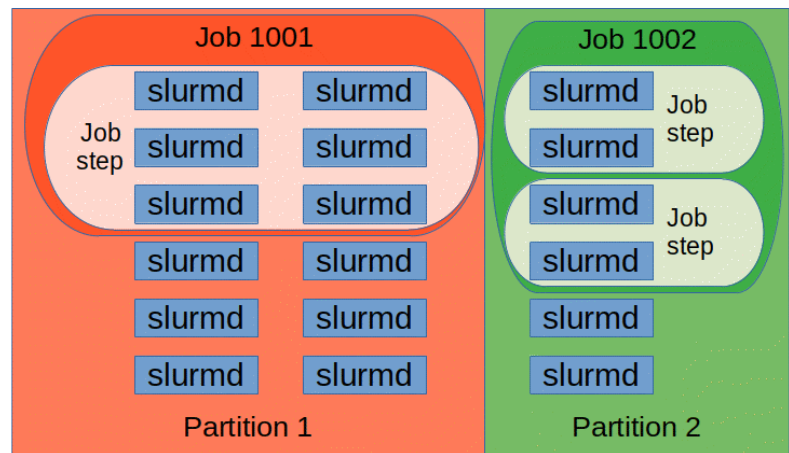
- 用戶提交的計算任務，記錄了資源需求、作業狀態、提交時間、執行時間、輸出檔案等資訊。
- 每個作業都有一個唯一的作業識別碼（Job ID），系統根據該識別碼追蹤作業進度。

Job steps

- 作業可以進一步細分為多個作業步驟，每個步驟通常對應一次 `srun` 命令的執行。
- 這允許同一個作業內部進行多階段或並行計算，並分別監控每個步驟的狀態與資源使用情形。

Tasks

- 一個job step可以包含一個或多個tasks，每個task代表一個要在計算節點上執行的工作。
- 在平行計算中，tasks通常用於執行同一個任務的多個實例，從而加速計算的進行。



Partition

晶創主機

佇列名稱	每個計劃最多 可用GPU總數	每個Job 最大執行時間	每個計畫同一時間		備註
			最多可執行 job的數量	最多可進到主機 排隊的Job數量	
dev	8	2 (小時)	2	2	H100
normal	16	48 (小時)	2	2	H100
normal2	16	48 (小時)	2	2	H200
4nodes	32	12 (小時)	2	2	H100

台灣杉二號

Queue 名稱	最長執行時間(小時)	高優先權	每位用戶最多 可提交計 算工作數上限	適用計畫	每位用戶最多 可取用 GPU 數上限
gp1d	24		20	各式計畫	40
gp2d	48		20	各式計畫	40
gp4d	96		20	各式計畫	80
gtest	0.5		5	各式計畫	40
express	96	v	20	企業與個人計畫	256

Partition

X86架構CPU計算環境

佇列名稱	可用核心數範圍	最長可執行時間 (hour)	每位用戶可同時執行工作數	每位用戶可排隊工作數	計算工作派送節點
vscode	1~112	2	1	1	限制從 Open OnDemand Portal派送，即 stn[01-02]
jupyter	1~112	2	1	1	
rstudio	1~112	2	1	1	
development	1~1120	8	1	1	
ct112	1~112	96	5	10	限制從登入節點、互動式繪圖節點派送，即 ilgpn[01-02] 與 intgpn[01-02]
ct448	113~448	96	4	8	
ct1k	449~1120	64 (更新前為96)	3	6	
ct2k	1121~2240	48	2	4	
ct4k	2241~4480	48	1	2	
ct8k	4481~8960	48	1	1	
visual-dev	1~112	2	1	1	限制從互動式繪圖節點派送，即 intgpn[01-02]
visual	1~448	48	1	1	

ARM架構CPU計算環境

佇列名稱	可用核心數範圍	最長可執行時間 (hour)	每位用戶可同時執行工作數	每位用戶可排隊工作數	計算工作派送節點
arm-dev	1~1440	2	1	1	限制從登入節點派送，即 ilgpn[01-02]
arm144	1~144	48	6	6	
arm576	145~576	48	4	4	
arm1440	577~1440	48	2	2	

<https://man.twcc.ai/@f1-manual/partition>

查看Partition的狀態

sinfo

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
gtest      up      30:00      2   idle gn[1001-1002]
gp1d       up  1-00:00:00    3 drain* gn[1010,1216,1226]
gp1d       up  1-00:00:00   20   mix  gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
gp1d       up  1-00:00:00   16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
gp2d*      up  2-00:00:00    3 drain* gn[1010,1216,1226]
gp2d*      up  2-00:00:00   20   mix  gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
gp2d*      up  2-00:00:00   16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
gp4d       up  4-00:00:00    3 drain* gn[1010,1216,1226]
gp4d       up  4-00:00:00   20   mix  gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
gp4d       up  4-00:00:00   16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
express    up  4-00:00:00    3 drain* gn[1010,1216,1226]
express    up  4-00:00:00   20   mix  gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
express    up  4-00:00:00   16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
```

sinfo -s

```
[u8880716@un-ln01 ~]$ sinfo -s
PARTITION AVAIL  TIMELIMIT  NODES(A/I/O/T)  NODELIST
gtest      up      30:00      0/2/0/2  gn[1001-1002]
gp1d       up  1-00:00:00   21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
gp2d*      up  2-00:00:00   21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
gp4d       up  4-00:00:00   21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
express    up  4-00:00:00   21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
```

sinfo -a 查看隱藏partition

查看Partition組態

scontrol show partition <partition_name>

```
[u8880716@cbi-lgn01 ~]$ scontrol show partition normal
PartitionName=normal
  AllowGroups=ALL DenyAccounts=gov112009,gov113008,gov113026,gov114029 AllowQos=ALL
  AllocNodes=ALL Default=NO QoS=p_normal
  DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=2 MaxTime=2-00:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
  Nodes=hgpn[02-05,17-21]
  PriorityJobFactor=10 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=1008 TotalNodes=9 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerNode=UNLIMITED MaxMemPerNode=UNLIMITED
  TRES=cpu=972,mem=17100000M,node=9,billing=972,gres/gpu=72
```

```
[u8880716@cbi-lgn01 ~]$ scontrol show partition -a taide
PartitionName=taide
  AllowGroups=ALL AllowAccounts=gov112009,gov113008,gov113026,gov113080 AllowQos=ALL
  AllocNodes=ALL Default=NO QoS=p_taide
  DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=YES
  MaxNodes=UNLIMITED MaxTime=3-00:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
  Nodes=hgpn[02-05,17-21]
  PriorityJobFactor=199 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=1008 TotalNodes=9 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerNode=UNLIMITED MaxMemPerNode=UNLIMITED
  TRES=cpu=972,mem=17100000M,node=9,billing=972,gres/gpu=72
```

查看計算節點組態

scontrol show node <node_name>

```
[u8880716@cbi-lgn01 ~]$ scontrol show node -a hgpn20
NodeName=hgpn20 Arch=x86_64 CoresPerSocket=56
CPUAlloc=1 CPUEfctv=108 CPUTot=112 CPULoad=17.90
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=gpu:H100:8(S:0-1)
NodeAddr=hgpn20 NodeHostName=hgpn20 Version=23.11.4
OS=Linux 4.18.0-513.24.1.el8_9.x86_64 #1 SMP Thu Mar 14 14:20:09 EDT 2024
RealMemory=1900000 AllocMem=1638400 FreeMem=242722 Sockets=2 Boards=1
CoreSpecCount=4 CPUSpecList=54-55,110-111
State=MIXED ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=GTAIDE
BootTime=2024-12-27T14:16:24 SlurmdStartTime=2025-02-07T15:53:54
LastBusyTime=2025-02-07T15:53:54 ResumeAfterTime=None
CfgTRES=cpu=108,mem=1900000M,billing=108,gres/gpu=8
AllocTRES=cpu=1,mem=1600G,gres/gpu=8 此行表示正在使用資源
CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/a ExtSensorsWatts=0 ExtSensorsTemp=n/a
```

此行表示正在使用資源
gres/gpu表示已使用的GPU數量

查詢有閒置GPU的節點

scontrol show node | grep -E 'NodeName|AllocTRES' | sed -n '/NodeName/{N; /gres\/gpu=8/!p}'

```
[u8880716@cbi-lgn01 ~]$ scontrol show node | grep -E 'NodeName|AllocTRES'
NodeName=hgpn02 Arch=x86_64 CoresPerSocket=56
AllocTRES=cpu=21,mem=800G,gres/gpu=7
NodeName=hgpn03 Arch=x86_64 CoresPerSocket=56
AllocTRES=cpu=16,mem=400G,gres/gpu=2
```


查看job

squeue

```
[u8880716@un-ln01 ~]$ squeue
      JOBID PARTITION      NAME      USER ST       TIME  NODES NODELIST(REASON)
      746268      gp4d  neb_in.n  u8714813 PD        0:00      26 (Resources)
      747428      gp4d  decompres u5727520 PD        0:00       1 (Priority)
      747019      gp2d    2DVVM  aaron900 R       6:15:32       1 gn1211
      747289      gp1d    bash  mattp131 R       3:20:11       1 gn1201
      744061      gp4d    sugar  caohieu9 R    1-01:58:36       1 gn1204
```

squeue -o "%.9i %.9P %.12j %.10u %.2t %.10M %.6D %.16R %b"

-o "... " 表示輸出格式

%b 表示GPU數量

```
[u8880716@un-ln01 ~]$ squeue -o "%.9i %.9P %.12j %.10u %.2t %.10M %.6D %.16R %b"
      JOBID PARTITION      NAME      USER ST       TIME  NODES NODELIST(REASON) TRES_PER_NODE
      741411      gp2d    dtpB    u5671665 PD        0:00       1 (QOSMaxGRESPerUs gpu:8
      741412      gp2d    bltB    u5671665 PD        0:00       1 (QOSMaxGRESPerUs gpu:8
      741413      gp2d    ls1mB   u5671665 PD        0:00       1 (QOSMaxGRESPerUs gpu:8
      741829      gp1d    bash    s6300121 R       45:28       1          gn1206 gpu:4
      741952      gp4d  bert_mlperf_ u8880716 R       26:30       2  gn[1206-1207] gpu:2
      741493      gp1d    run.sh   u8144818 R       3:01:33       1          gn1204 gpu:1
      741491      gp1d    run.sh   u8144818 R       3:04:08       1          gn1204 gpu:1
```


查看job history

```
sacct --user <user_name> -X -S 2025-06-01 -E 2025-07-10 -o  
JobID,Account,User%10,Partition%10,NNodes,AllocTRES%40,State%20,Submit,Elapsed
```

-X 忽略job step

-S 起始時間

-E 結束時間

-o 輸出格式

```
[u8880716@un-ln01 ~]$ sacct --user u8880716 -X -S 2025-06-01 -E 2025-07-10 -o JobID,Account,User%10,Partition%10,NNodes,AllocTRES%40,State%20,Submit,Elapsed
```

JobID	Account	User	Partition	NNodes	AllocTRES	State	Submit	Elapsed
723358	gov108032	u8880716	gp1d	1	billing=4,cpu=4,gres/gpu=8,mem=720G,nod+	COMPLETED	2025-06-16T14:54:56	08:38:56
723532	gov108032	u8880716	gp1d	1	billing=4,cpu=4,gres/gpu=8,mem=720G,nod+	CANCELLED by 12409	2025-06-17T09:54:24	00:56:04
723540	gov108032	u8880716	gp1d	1	billing=4,cpu=4,gres/gpu=8,mem=720G,nod+	COMPLETED	2025-06-17T11:06:36	04:06:23
723637	gov108032	u8880716	gp1d	1	billing=4,cpu=4,gres/gpu=8,mem=720G,nod+	COMPLETED	2025-06-17T17:15:21	02:20:41
728175	gov109092	u8880716	gp4d	1	billing=4,cpu=4,gres/gpu=8,mem=720G,nod+	COMPLETED	2025-07-08T17:25:13	14:06:37

執行方式

■ 腳本

- sbatch <slurm script>

■ 互動式

- srun：以一行指令取得資源與執行運算
- salloc：先取得資源，進入計算節點，在命令列上逐一執行運算指令，最後關閉job

Submit slurm script

run.slurm

```
#!/bin/bash
#SBATCH --account=<PROJECT_ID>      # (-A) iService Project ID
#SBATCH --job-name=test              # (-J) Job name
#SBATCH --partition=dev              # (-p) Slurm partition
#SBATCH --nodes=2                    # (-N) Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4            # (-c) Number of cores per MPI task
#SBATCH --ntasks-per-node=1          # Maximum number of tasks on each node
#SBATCH --gres=gpu:2                 #
#SBATCH --time=0-00:30:00            # (-t) Wall time limit (days-hrs:min:sec)
#SBATCH --output=%x-%j.out           # (-o) Path to the standard output file, %x is job name, %j is job id
#SBATCH --error=%x-%j.err            # (-e) Path to the standard error file
#SBATCH --mail-type=END,FAIL          # Mail events (NONE, BEGIN, END, FAIL, ALL)
#SBATCH --mail-user=user@example.com # Where to send mail. Set this to your email address

module purge
module load intel/....

SIF="/path/openmpi.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

srun --mpi=pmix -n 4 $SINGULARITY ./hello
mpiexec ./hello
```

--cpus-per-task=4 for Taiwania2
--cpus-per-task=1~12 for Nano5

Submit and manager job

提交Job Script

- sbatch run.slurm

查看使用者的job 狀態

- squeue -u <username>
- squeue -j <job_id>

查看正在運行的Job 組態

- scontrol show job <job_id>

刪除 job

- scancel <job_id>

srun

On Forerunner1

```
srun -A <project> -p development --ntasks-per-node=1 <command>
```

On Nano5

```
srun -A <project> -p dev --gres=gpu:1 <command>
```

On Taiwania2

```
srun -A <project> -p gtest --nodes 1 --ntasks-per-node=1 --gres=gpu:1 <command>
```

```
[u8880716@cbi-lgn01 ~]$ srun -A GOV108032 -J test -p dev --gres=gpu:1 nvidia-smi
Fri Jul 25 15:15:02 2025
```

NVIDIA-SMI 550.127.08				Driver Version: 550.127.08				CUDA Version: 12.4			
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Memory-Usage	Volatile	Uncorr.	ECC		
Fan	Temp		Pwr:Usage/Cap				GPU-Util	Compute M.	MIG M.		
0	NVIDIA H100	80GB HBM3	On	00000000:86:00.0	Off				0		
N/A	26C	P0	70W / 700W	1MiB / 81559MiB			0%	Default	Disabled		

```
Processes:
GPU  GI  CI  PID  Type  Process name  GPU Memory
   ID  ID               Usage
=====
No running processes found
```

salloc

```
salloc -A <project> -p gtest --nodes 1 --ntasks-per-node 1 --gres=gpu:1
```

```
Echo $SLURM_NODELIST # check allocated compute node
```

```
ssh <compute_node>
```

```
<command> ...
```

```
scancel <job_ID>
```

```
[u8880716@un-ln01 ~]$ salloc -A GOV108032 -p gtest --nodes 1 --ntasks-per-node 1 --gres=gpu:1
WARNING: When login node is DOWN, Your Job will FAIL!
WARNING: Please use `sbatch` instead :)
salloc: Granted job allocation 753642
salloc: Waiting for resource configuration
salloc: Nodes gn1002 are ready for job
[u8880716@un-ln01 ~]$ echo $SLURM_NODELIST
gn1002
[u8880716@un-ln01 ~]$ scancel 753642
salloc: Job allocation 753642 has been revoked.
[u8880716@un-ln01 ~]$ squeue -u u8880716
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
-------	-----------	------	------	----	------	-------	------------------

Slurm環境變數

執行job運算，產生Slurm的環境變數，可以做為平行運算的參數

以下是常用的Slurm環境變數

變數名稱	描述
SLURM_JOB_ID	Job ID
SLURM_JOB_NAME	Job name
SLURM_JOB_ACCOUNT	Project id
SLURM_JOB_NUM_NODES SLURM_NNODES	Number of nodes allocated to job
SLURM_JOB_NODELIST SLURM_NODELIST	Nodes assigned to job
SLURM_NTASKS_PER_NODE	Number of tasks requested per node.
SLURM_NTASKS	Total tasks in job
SLURM_GPUS_ON_NODE	Number of GPUs per node. (Only on Nano5)

列出所有Slurm環境變數

```
srun -A <ProjectID> -p normal --gres=gpu:2 printenv | grep SLURM
```

```
srun: job 81496 queued and waiting for resources
srun: job 81496 has been allocated resources
SLURM_CONF=/etc/slurm/slurm.conf
SLURM_PRIO_PROCESS=0
SLURM_UMASK=0022
SLURM_CLUSTER_NAME=hpc
SLURM_SUBMIT_DIR=/home/u8880716/llama_factory_b842457e
SLURM_SUBMIT_HOST=cbi-lgn01
SLURM_JOB_NAME=printenv
SLURM_JOB_CPUS_PER_NODE=1
SLURM_MEM_PER_NODE=409600
SLURM_NTASKS=1
SLURM_NPROCS=1
SLURM_DISTRIBUTION=cyclic,pack
SLURMD_DEBUG=2
SLURM_JOB_ID=81496
SLURM_JOBID=81496
SLURM_STEP_ID=0
SLURM_STEPID=0
SLURM_NNODES=1
SLURM_JOB_NUM_NODES=1
SLURM_NODELIST=hgpn03
SLURM_JOB_PARTITION=normal
SLURM_TASKS_PER_NODE=1
SLURM_JOB_UID=12409
SLURM_JOB_USER=u8880716
SLURM_JOB_GID=18361
SLURM_JOB_GROUP=TRI1072239
```

```
SLURM_JOB_ACCOUNT=gov108032
SLURM_JOB_QOS=normal
SLURM_JOB_NODELIST=hgpn03
SLURM_STEP_NODELIST=hgpn03
SLURM_STEP_NUM_NODES=1
SLURM_STEP_NUM_TASKS=1
SLURM_STEP_TASKS_PER_NODE=1
SLURM_STEP_LAUNCHER_PORT=43051
SLURM_SRUN_COMM_PORT=43051
SLURM_SRUN_COMM_HOST=172.21.101.1
SLURM_GPUS_ON_NODE=2
SLURM_STEP_GPUS=0,4
SLURM_TOPOLOGY_ADDR=ibsw1.hgpn03
SLURM_TOPOLOGY_ADDR_PATTERN=switch.node
SLURM_CPUS_ON_NODE=1
SLURM_CPU_BIND=quiet,mask_cpu:0x000000000000001000000000000000
SLURM_CPU_BIND_LIST=0x000000000000001000000000000000
SLURM_CPU_BIND_TYPE=mask_cpu:
SLURM_CPU_BIND_VERBOSE=quiet
SLURM_JOB_END_TIME=1766045527
SLURM_JOB_START_TIME=1765872727
SLURM_TASK_PID=2708572
SLURM_NODEID=0
SLURM_PROCID=0
SLURM_LOCALID=0
SLURM_LAUNCH_NODE_IPADDR=172.21.101.1
SLURM_GTIDS=0
SLURMD_NODENAME=hgpn03
```


NIAR 國家實驗研究院
NATIONAL INSTITUTES OF APPLIED RESEARCH

Nano5

Taiwania 2

Forerunner 1

```
[u880716@ilgn01 ~]$ module avail

----- /pkg/modulefiles/software -----
apps/abaqus/2023
apps/abaqus/2024
apps/adf/2024.101-intelmpi-intel (D)
apps/adf/2024.102-intelmpi-intel (D)
apps/ansoft/v2023r2
apps/ansys/CFX/v2020r2
apps/ansys/CFX/v2022r2
apps/ansys/CFX/v2023r2
apps/ansys/CFX/v2024r1
apps/ansys/Fluent/v2020r2
apps/ansys/Fluent/v2022r2
apps/ansys/Fluent/v2023r2
apps/ansys/Fluent/v2024r1 (D)
apps/ansys/ICEMCFD/v2020r2
apps/ansys/ICEMCFD/v2022r2
apps/ansys/ICEMCFD/v2023r2
apps/ansys/ICEMCFD/v2024r1 (D)
apps/ansys/workbench_APDL/v2020r2
apps/ansys/workbench_APDL/v2022r2
apps/ansys/workbench_APDL/v2023r2
apps/ansys/workbench_APDL/v2024r1 (D)
apps/biovia/ms24
apps/biovia/ms25 (D)
apps/chem/castep-24.1
apps/chem/gromacs/2023.4
apps/chem/gromacs/2023.4/fftw/3.3.10
apps/chem/gromacs/2023.4/hwloc
apps/chem/gromacs/2023.4/openmpi/5.0.1
apps/chem/gromacs/2023.4/ucc/1.3.x
apps/chem/gromacs/2023.4/ucx/1.16
apps/cmp/BigDFT/1.9.5.25
apps/gaussian/g16

apps/lammps/lammps-2AUG2023-SHIK-potential
apps/lammps/lammps-2AUG2023
apps/lsdyna/R12.0.0
apps/lsdyna/R13.0.0
apps/lsdyna/R13.1.0 (D)
apps/matlab/R2023a
apps/matlab/R2024b (D)
apps/molpro/2024.1
apps/openfoam/OpenFOAM-v2312-x86_64-Intel_OneAPI_AVX512
apps/openfoam/OpenFOAM-v2506-ARM64-GCC
apps/openfoam/OpenFOAM-v2506-x86_64-Intel_OneAPI_AVX512 (D)
apps/opensees/3.4.0-gcc
apps/opensees/3.4.0-gcc
apps/opensees/3.5.0-gcc
apps/opensees/3.6.0-icc
apps/opensees/3.7.1-gcc
apps/opensees/3.7.1-icc (D)
apps/opensees/3.7.1
apps/orca/6.0.1
apps/qchem/6.2.0
apps/starccmp/STAR-CCM_Plus_v2506
libs/eigen/3.4.0
libs/hdf5/1.14.3
libs/scalapack/2.2.0-intel-oneapi-2023.2
tools/code-server
tools/jupyterlab
tools/miniconda3
tools/paraview/6.0.1
tools/paraview/client/5.12.0
tools/paraview/server/5.12.0
tools/rstudio/4.4.0
tools/singularity-ce/4.1.1

----- /pkg/modulefiles/middleware/compiler -----
gcc/8.5.0 (D) gcc/11.2.0 intel/2020_u4 intel/2022_3_1 intel/2024_01_46 (D)
gcc/10.4.0 intel/2019_u5 intel/2021_4 intel/2023_2
```

Modules command

查看可安裝的軟體 module avail

載入軟體 module load <package>

查看載入軟體 module list

清除軟體 module purge

Singularity container

- Singularity 是一種專為高效能運算 (HPC) 環境設計的容器化技術，其主要目的是將應用程式及其所有依賴項（如程式庫、環境設定等）打包成一個獨立的容器映像檔，從而實現跨平台、跨叢集運算環境的一致性與可重現性。
- **環境封裝**：Singularity 可將一個完整的運行環境打包成單一映像檔(*.sif)，使用者無需在每個運算節點上手動安裝軟體與依賴，就能確保應用程式在不同系統間執行時具有一致性。
- **跨平台可攜**：使用者可以在個人電腦上建立容器，並將其部署至大規模 HPC 叢集、雲端或其他運算平台，確保相同程式碼在不同環境中得到一致的執行結果。
- **權限一致**：Singularity 的設計理念是“用戶在容器內的身份與在宿主(Host)系統上相同”，因此不需要 root 權限來執行容器，降低了安全風險，也符合多使用者共享環境的要求。
- **輕量化部署**：與 Docker 等容器技術相比，Singularity 不依賴於守護進程（daemon），能更好地與 HPC 系統的資源調度器（例如 Slurm）整合，使得容器化工作流程能夠順利提交到叢集任務中執行。
- **MPI 與 GPU 支援**：Singularity 能夠透過綁定(binding)主機上的網路設備、MPI 函式庫以及 GPU 等硬體資源，使得容器內的應用能夠利用 HPC 叢集的高速網路與專用加速器，達到接近裸機的效能。
- **再現性**：Singularity 可確保運行環境與軟體版本固定，使得實驗結果更具再現性，利於學術交流與結果驗證。

執行Singularity容器

1. 從Docker hub下載映像檔

- `singularity pull pytorch_24.05-py3.sif docker://nvcr.io/nvidia/pytorch:24.05-py3`

2. 運行容器

- 背景執行 `singularity exec/run --nv <image.sif> <command>`

- 互動執行 `singularity run/shell --nv <image.sif>`

```
$ singularity shell instance:///instance3
```

```
Singularity>
```

- `--nv` 允許容器使用GPU

- 引用/work目錄 `singularity exec/run -B /work:/work --nv <image.sif> <command>`
 - 已預設綁定home目錄，不用設定 `-B /home:/home`

- 列出容器內的python套件

- `singularity exec/run --nv <image.sif> pip list`

建立客製化容器映像檔

1. 建立客製化容器映像檔需要sudo權限
2. 另尋有sudo權限Linux電腦 (例如TWCC VM, WSL, VirtualBox)建立
客製化容器映像檔
3. 安裝相關依賴庫、Go語言與Singularity
 - 注意Go語言與Singularity版本有搭配
4. 實作版本
 - Linux Ubuntu 24.04
 - Docker 28.3.2 and Nvidia container toolkit 1.17.8 (非必要)
 - Go 1.23.6
 - Singularity: Apptainer 1.4.1
 - <https://github.com/apptainer/apptainer/blob/main/INSTALL.md>

Install Go & Singularity

```
# Ensure repositories are up-to-date
sudo apt-get update
# Install debian packages for dependencies
sudo apt-get install -y \
    build-essential \
    libseccomp-dev \
    uidmap \
    fakeroot \
    cryptsetup \
    tzdata \
    dh-apparmor \
    libsubid-dev \
    pkg-config \
    curl wget git

# Install GO
export GOVERSION=1.23.6 OS=linux ARCH=amd64 # change this as you need

wget -O /tmp/go${GOVERSION}.${OS}-${ARCH}.tar.gz \
    https://dl.google.com/go/go${GOVERSION}.${OS}-${ARCH}.tar.gz

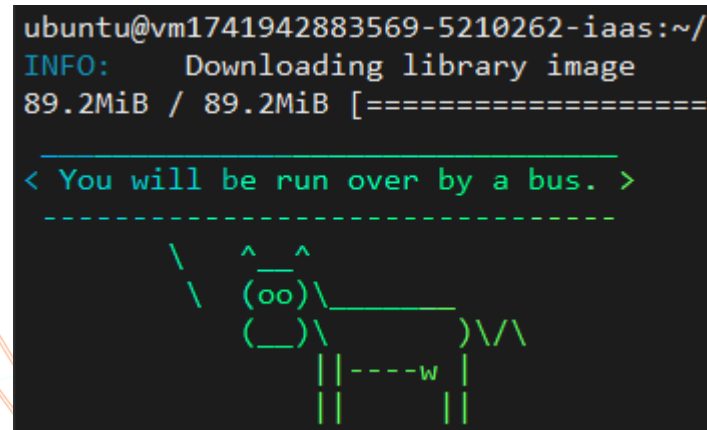
sudo tar -C /usr/local -xzf /tmp/go${GOVERSION}.${OS}-${ARCH}.tar.gz

# add /usr/local/go/bin to the PATH environment variable
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc
```

```
# Install singulairty
git clone https://github.com/apptainer/apptainer.git
cd apptainer
git checkout v1.4.1

# Compiling Singularity
./mconfig
cd $(/bin/pwd)/builddir
make -j 4
sudo make install

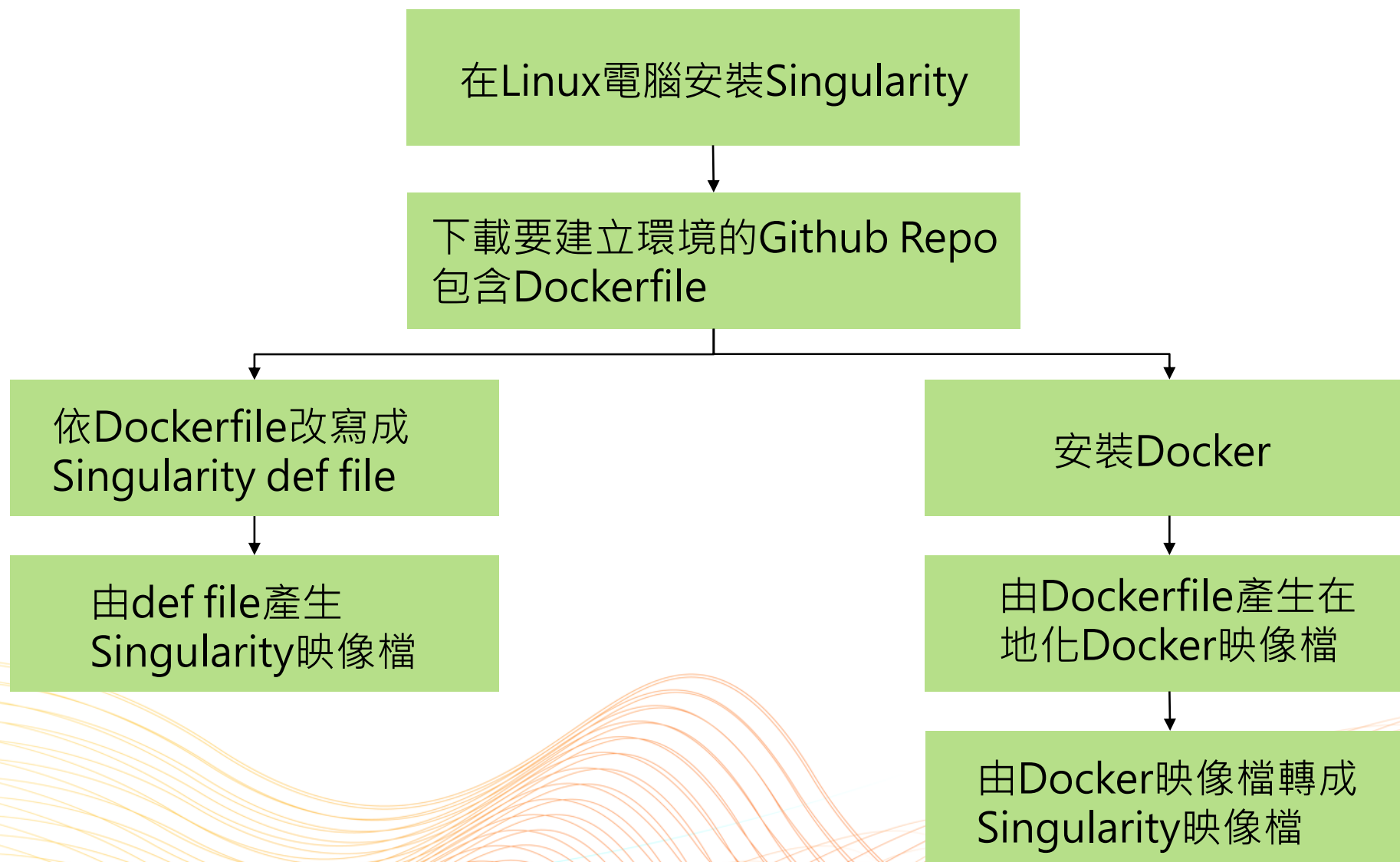
singularity --version
singularity run library://godlovedc/funny/lolcow
```



```
ubuntu@vm1741942883569-5210262-iaas:~/
INFO:   Downloading library image
89.2MiB / 89.2MiB [=====]

< You will be run over by a bus. >
-----
      ^__^
      (oo)\_______
      (__)\       )\/\
          ||----w |
          ||     ||
```

建立客製化容器映像檔的步驟



建立定義檔Singularity Definition file

vi <definition>.def

```
Bootstrap: library
From: ubuntu:18.04
Stage: build

%setup
    touch /file1
    touch ${SINGULARITY_ROOTFS}/file2

%files
    /file1
    /file1 /opt

%environment
    export LISTEN_PORT=12345
    export LC_ALL=C

%post
    apt-get update && apt-get install -y netcat
    NOW=`date`
    echo "export NOW=\"${NOW}\"" >> $SINGULARITY_ENVIRONMENT

%runscript
    echo "Container was created $NOW"
    echo "Arguments received: $*"
    exec echo "$@"
```

從Docker hub取得映像檔

建立主機端和容器的檔案、資料夾

複製主機端的檔案、資料夾到容器

設定環境變數

執行安裝軟體指令

啟動容器時，執行程式

https://docs.sylabs.io/guides/3.7/user-guide/definition_files.html
HowTo : 建立 TWNIA2 容器

範例：Tensorflow for H100

Bootstrap: docker

From: nvcr.io/nvidia/tensorflow:23.10-tf2-py3

%help

此容器用於 CosmoFlow 資料預處理，解決了：

1. Protobuf 版本不相容問題 (固定於 3.20.3)
2. H100 GPU (Compute 9.0) 支援
3. MPI 與 UCX 通訊優化

%post

更新系統並安裝必要工具

```
apt-get update && apt-get install -y --no-install-recommends \  
    build-essential \  
    libopenmpi-dev \  
    git
```

強制降級 Protobuf 以符合舊版 TensorFlow 邏輯，避免 TypeError

```
pip install --no-cache-dir --upgrade pip  
pip install --no-cache-dir "protobuf<3.21,>=3.20.3"
```

安裝並行計算與資料處理必要套件

```
pip install --no-cache-dir mpi4py numpy
```

%environment

解決 UCX 速度辨識錯誤的環境變數預設值

```
export UCX_TLS=tcp,self,sm
```

```
export UCX_IB_RETRY_COUNT=20
```

確保 Protobuf 使用 C++ 實作以獲得最佳效能

```
export PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=cpp
```

%runscript

```
exec python "$@"
```

建立映像檔

方法一：By singularity definition file

```
sudo singularity build <image>.sif <definition>.def
```

方法二：By docker local image

1. `docker build -t <repository>:<tag> .`
2. `sudo singularity build <image>.sif docker-daemon://<repository>:<tag>`

方法三：By docker local image and compressed file

1. `docker build -t <repository>:<tag> .`
2. `docker save <repository>:<tag> -o <image>.tar`
3. `sudo singularity build <image>.sif docker-archive://$(pwd)/<name>.tar`

```
ubuntu@vm1692770172744-3864139-iaas:~$ docker image list
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mlperf-nvidia	language_model	7b697e4dcb2b	24 hours ago	22.5GB
llama-factory	latest	a334a786d902	3 months ago	22.7GB
nvcr.io/nvidia/pytorch	24.01-py3	8470a68886ff	7 months ago	22GB
nvcr.io/nvidia/pytorch	23.04-py3	b4428941db4f	16 months ago	20.4GB

範例

On TWCC VM (製作映像檔的環境)

1. `cd <path/to/Dockerfile>`
2. `docker build -t <repository>:<tag> .`
3. `sudo singularity build <image>.sif docker-daemon://<repository>:<tag>`
4. 傳送映像檔<image>.sif 到晶創25主機

```
ubuntu@vmcpuv-5372004-iaas:~$ cd bert_mlperf_3.0
ubuntu@vmcpuv-5372004-iaas:~/bert_mlperf_3.0$ ls
a30-run_and_time.sh      Dockerfile
a30.sub                  extract_features.py
bmm1.py                  file_utils.py
```

```
ubuntu@vmcpuv-5372004-iaas:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
bert_mlperf   pytorch-2304  dd6dc34e6fc8  33 minutes ago  22.5GB
ubuntu@vmcpuv-5372004-iaas:~$ ls
bert  bert_mlperf_3.0  bert_mlperf_pytorch_2304.sif  cosmo_flow_tf.def
```

On Nano5

1. 在Slurm script引用Singularity 映像檔

```
#!/bin/bash
#SBATCH -A GOV108032      # iService Project id GOV112012 GOV110171
#SBATCH -J H100x8_bs16   # job name
#SBATCH -p normal        # partition
#SBATCH --ntasks-per-node=1 # Number of MPI tasks (i.e. processes)
#SBATCH --cpus-per-task=4  # Number of cores per MPI task
#SBATCH --nodes=1         # Maximum number of nodes to be allocated
#SBATCH --gres=gpu:8
#SBATCH -o %x_%j.out      # Path to the standard output file

export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export OMP_NUM_THREADS=1

SECONDS=0
SIF=/work/$(whoami)/sif/bert_mlperf_pytorch_2304.sif
SINGULARITY="singularity run -B /work:/work --nv $SIF"
```

安裝軟體

- 編譯語言：c/c++, fortran, CUDA
 - 透過Lmod引用gcc, cmake, intel oneAPI
 - 編譯原始碼

- 直譯語言：python
 - 透過Lmod引用miniconda
 - pip install <package>

1. 在登入節點，編譯原始碼

1. 引用編譯器

```
module load gcc, intel
```

2. 製作makefile

```
mkdir build
```

```
cd build
```

```
cmake -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++ -DCMAKE_C_FLAGS="-g -O3" ..
```

3. 編譯makefile

```
make -j
```

2. 執行運算的腳本

```
#SBATCH ...
```

```
module purge
```

```
module gcc
```

```
module load intel
```

```
mpirun -n $SLURM_NTASKS <application> ...
```


Miniconda

1. 在登入節點，預先安裝環境

1. 引用Miniconda

```
module load miniconda3/24.11.1
```

2. 建立虛擬環境

```
conda create <env_name> python=3.x -y
```

3. 進入虛擬環境

```
conda activate <env_name>
```

4. 安裝套件

```
conda install <package>
```

```
pip install <package>
```

安裝套件路徑 ~/.conda/envs/<env_name>/lib/python3.x/site-packages/...

5. 退出虛擬環境

```
conda deactivate
```

2. 執行運算的腳本

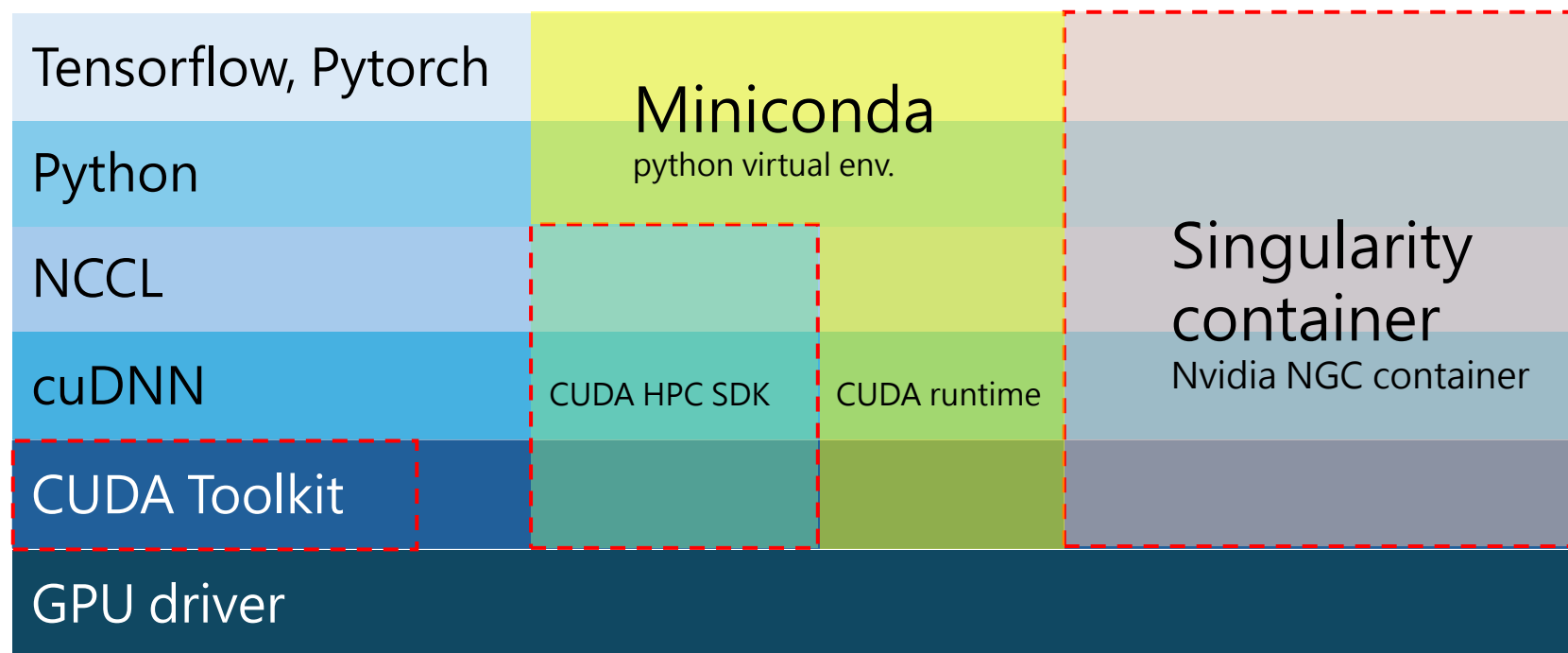
```
#SBATCH ...

module purge
module load miniconda3/24.11.1
conda activate <env_name>

python train_script.py ...

conda deactivate
```

深度學習訓練的軟體架構 on Nano5



CUDA Toolkit (NVCC compiler, CUDA runtime, cuBLAS, cuFFT, cuSPARSE)
module 軟體管理有提供

Nvidia NGC container

<https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-25-11.html>

The screenshot shows the NVIDIA Optimized Frameworks documentation page. The header includes the NVIDIA logo and a search bar. The main content area is titled "PyTorch Release 25.11" and includes a link to the NVIDIA container image. The left sidebar contains a navigation menu with categories like "Topics", "NVIDIA Optimized Frameworks", "Support Matrix", and "Optimized Frameworks Release Notes". The "PyTorch Release 25.11" section is expanded, showing the "Contents of the PyTorch container" and a list of included libraries and tools.

NVIDIA Optimized Frameworks

Search NVIDIA Optimized Frameworks

Topics

- NVIDIA Optimized Frameworks
- Support Matrix
- Optimized Frameworks Release Notes
 - CUDA DL Release Notes
 - TensorRT Release Notes
 - DGL Release Notes
 - JAX Release Notes
 - Kaldi Release Notes
 - NVIDIA Optimized Deep Learning Framework, powered by Apache MXNet Release Notes
 - PyTorch Release Notes

NVIDIA Docs Hub Homepage > NVIDIA Optimized Frameworks > NVIDIA Optimized Frameworks > PyTorch Release 25.11

PyTorch Release 25.11

Is this page helpful?

The NVIDIA container image for this PyTorch release is available on [NGC](#).

Contents of the PyTorch container

This container image contains the complete source of the version of PyTorch in `/opt/pytorch`. It is prebuilt and installed in the default Python environment (`/usr/local/lib/python3.12/dist-packages/torch`) in the container image.

The container also includes the following:

- Please refer to CUDA section for the list of libraries inherited from CUDA container.
- NVIDIA CUDA 13.0.2.006**
- Torch-TensorRT 2.10.0a0**
- NVIDIA DALI® 1.52**
- nvImageCodec 0.2.0.7**
- MAGMA 2.6.2**
- JupyterLab 4.3.6** including [Jupyter-TensorBoard](#)
- TensorRT Model Optimizer 0.37
- TransformerEngine 2.9**
- NVIDIA cuBLASmp 0.6.0.84**

CHAT

The screenshot shows the NVIDIA NGC Catalog page for PyTorch. The header includes the NVIDIA logo and a search bar. The main content area is titled "NVIDIA PyTorch" and includes a description of the framework. The left sidebar contains a navigation menu with categories like "Topics", "NVIDIA Optimized Frameworks", "Support Matrix", and "Optimized Frameworks Release Notes". The "PyTorch Release 25.11" section is expanded, showing the "Contents of the PyTorch container" and a list of included libraries and tools.

NVIDIA PyTorch

PyTorch is a GPU accelerated tensor computational framework. Functionality can be extended with common Python libraries such as NumPy and SciPy. Automatic differentiation is done with a tape-based system at the functional and neural network layer levels.

NVIDIA AI Enterprise Supported

Get Container

25.11-py3 Scanned Signed

Overview Tags Layers Security Scanning Related Collections

A public key is required to validate the signed images below. View all public keys maintained by NVIDIA

119 Tags Search within 119 tags...

25.11-py3	Scanned	Signed	11/26/2025 8:22 AM	8.85 GB	2 Architectures	<code>nvcr.io/nvidia/pytorch:25.11-py3</code>
25.11-py3-igpu	Scanned	Signed	11/26/2025 8:20 AM	4.42 GB	1 Architecture	<code>nvcr.io/nvidia/pytorch:25.11-py3-igpu</code>

主機使用實作示範

1. 登入主機，密碼 + OTP驗證

2. 查詢資源

1. 查看計劃錢包: `wallet <Project_ID>`

2. 查看partition: `sinfo`

3. 查看job狀態: `squeue`

```
squeue -o "%.9i %.9P %.12j %.10u %.2t %.10M %.6D %.16R %b"
```

4. 查看空閒GPU的節點 : `scontrol show node | grep -E 'NodeName|AllocTRES' | sed -n '/NodeName/{N; /gres\/gpu=8/!p}'`

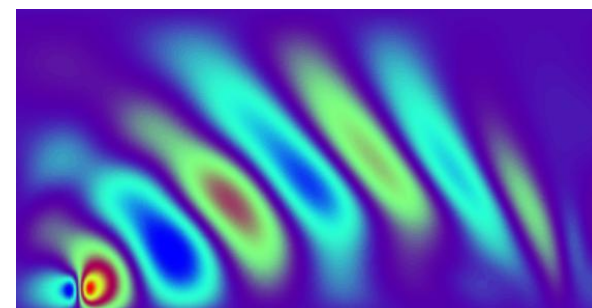
5. 查看HFS容量 : `hfsquota (N5)`

```
/usr/lpp/mmfs/bin/mm1squota --block-size auto fs01 fs02 (T2)
```


miniWeather

miniWeather

- miniWeather 是一個模擬大氣天氣與氣候基本動力學的小型數值模式。它主要用於展示並行程式設計技術，並適用於多種運算平台，如 CPU 和 GPU。該程式透過簡化的氣象模型，使使用者能夠專注於高效能計算和並行計算技術的應用。
- **物理現象**：miniWeather 模擬乾燥、可壓縮的分層非靜力流。
- **高效能運算支援**：
 - 可運行於多種架構，包括 CPU、GPU 以及高效能運算集群（HPC）。
 - 透過並行程式設計提升計算效率。
 - 支援 **MPI** / OpenMP / OpenACC / Fortran
- **應用領域**
 - 氣象模擬：研究小尺度大氣現象的行為。
 - 高效能計算（HPC）訓練：示範如何利用 MPI/OpenMP 進行並行程式開發。
 - GPU 運算



Mountain
Gravity Waves

<https://github.com/mrnorman/miniWeather>

程式安裝 (1/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體

✓ parallel-netcdf 軟體

<https://parallel-netcdf.github.io/wiki/Download.html>

✓ 下載 parallel-netcdf 軟體

\$ wget <https://parallel-netcdf.github.io/Release/pnetcdf-1.14.1.tar.gz>

\$ tar -zxvf pnetcdf-1.14.1.tar.gz

\$ cd pnetcdf-1.14.1

\$./configure --prefix=\$HOME/opt

\$ make -j8

\$ make install

```
$ ./configure --prefix=$HOME/opt
PnetCDF Version 1.14.1

Features:  Build static libraries      - yes
           Build shared libraries       - yes
           Build Fortran APIs           - yes
           Build C++ APIs               - yes

Compilers: MPICC      = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpicc
           MPICXX     = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpicxx
           MPIF77      = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpif77
           MPIF90      = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpif90
           CFLAGS      = -g -O2
           CXXFLAGS    = -g -O2
           FFLAGS      = -g -O2
           FCFLAGS     = -g -O2
           SEQ_CC      = /usr/bin/gcc
```

成功

程式安裝 (2/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

② 安裝 miniWeather 軟體

✓ 下載 miniWeather 軟體

```
$ cd $HOME
```

```
$ git clone https://github.com/mrnorman/miniWeather.git
```

```
$ cd miniWeather
```

```
$ git submodule update --init --recursive
```

✓ 編譯

```
$ cd c
```

```
$ mkdir build-F1
```

```
$ cd build-F1
```

```
$ cp ../build/cmake_clean.sh .
```

程式安裝 (3/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

② 安裝 miniWeather 軟體

✓ 編譯 (Cont.)

建立 cmake_intel2024.sh

```
#!/bin/bash
module load intel/2024_01_46
export TEST_MPI_COMMAND="mpirun -n 1"
unset CUDAFLAGS
unset CXXFLAGS
PARALLEL_NETCDF_ROOT=$HOME/opt

./cmake_clean.sh

cmake -DCMAKE_CXX_COMPILER=mpicxx \
      -DCXXFLAGS="-Ofast -march=native -mtune=native -DNO_INFORM -std=c++11 -I${PARALLEL_NETCDF_ROOT}/include" \
      -DLDFLAGS="-L${PARALLEL_NETCDF_ROOT}/lib -lpnetcdf" \
      -DOPENMP_FLAGS="-fopenmp" \
      -DNX=200 \
      -DNZ=100 \
      -DDATA_SPEC="DATA_SPEC_GRAVITY_WAVES" \
      -DSIM_TIME=1500 \
      ..
```


程式安裝 (4/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

② 安裝 miniWeather 軟體

✓ 編譯 (Cont.)

```
$ bash cmake_intel2024.sh
```

```
$ make
```

```
$ ls
```

CMakeCache.txt	CMakeFiles	cmake_intel2024.sh	Makefile	mpi_test	openmp_test	serial_test
cmake_clean.sh	cmake_install.cmake	CTestTestfile.cmake	mpi	openmp	serial	

✓ 建立工作目錄

```
$ cd $HOME/miniWeather
```

```
$ mkdir work-dir
```

```
$ cd work-dir
```

```
$ cp ../c/build-F1/mpi.
```

程式安裝 (5/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

② 安裝 miniWeather 軟體

③ 執行 2node-4MPI

✓ 建立批次作業檔案 job-2node-4mpi

```
#!/bin/bash
#SBATCH --account=project_id      # (-A) iService Project ID
#SBATCH --job-name=p2node4mpi     # (-J) Job name
#SBATCH --partition=development   # (-p) Slurm partition development ct112
#SBATCH --nodes=2                 # (-N) Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=1         # (-c) Number of cores per MPI task
#SBATCH --ntasks-per-node=2       # Maximum number of tasks on each node
#SBATCH --output=zzz-2node-4MPI-%j.out # outputfile
export OMP_NUM_THREADS=1
export LD_LIBRARY_PATH=$HOME/opt/lib:$LD_LIBRARY_PATH
export PATH=$HOME/opt/bin:$PATH
ml intel/2024_01_46
srun --mpi=pmix_v4 ./mpi
```

\$ batch job-2node-4mpi

安裝 ncview 繪圖軟體

- ① 建立新工作環境 ncview_env，並且安裝 ncview

```
$ ml tools/miniconda3
```

```
$ conda create -n ncview_env -c conda-forge ncview -y
```

```
#  
# To activate this environment, use  
#  
#   $ conda activate ncview_env  
#  
# To deactivate an active environment, use  
#  
#   $ conda deactivate
```

登入互動繪圖節點後，進入 ncview_env 工作環境

```
$ ml tools/miniconda3
```

```
$ conda activate ncview_env
```

```
(ncview_env) $ ncview output.nc
```

Activities Ncview 2.1.8

Ncview 2.1.8

Ncview 2.1.8 David W. Pierce 8 March 2017

variable=wwnd

frame 141/151

displayed range: -0.266445 to 0.573243

Current: (i=142, j=95) 0.0145774 (x=142, y=95)

Quit ->1 << < || > >> Edit ? Delay: Opts

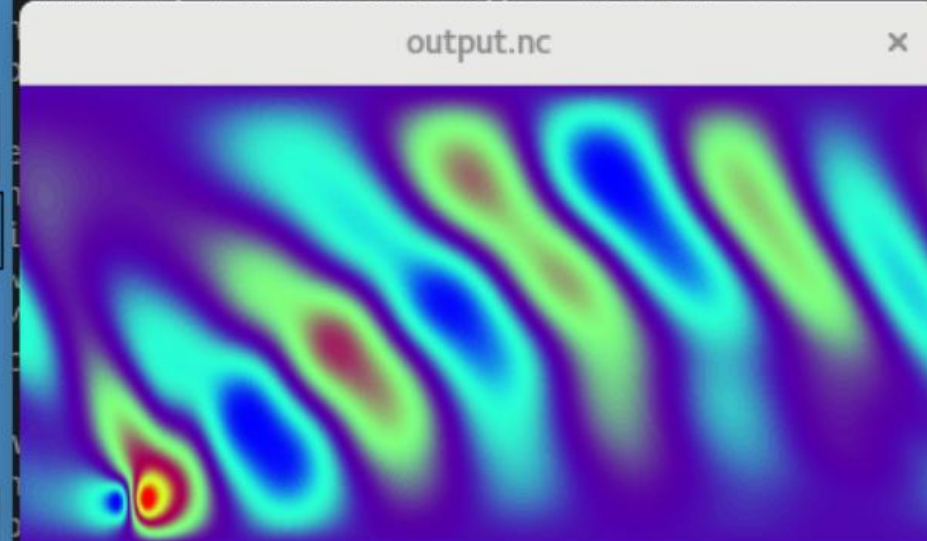
3gauss Inv P Inv C M X2 Linear Axes Range Bi-lin Print

Var: dens uwnd **wwnd** theta

Dim:	Name:	Min:	Current:	Max:	Units:
Scan:	t	0	1400	1500	-
Y:	z	0	-Y-	99	-
X:	x	0	-X-	199	-

018/20260303F1-Training-Course/miniWeather/work-dir

```
nd...
icit kill or server shutdown).
work-dir]$ ncview output.nc
March 2017
ce/ncview_home_page.html
David W. Pierce
WARRANTY; for details type `ncview -w'.
```



```
e found, using internal defaults
nd...
```



Pytorch DDP

DDP, Distributed Data Parallelism

Use DDP on Pytorch

```
import torch.multiprocessing as mp # 啟用多進程
from torch.utils.data.distributed import DistributedSampler
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.distributed import init_process_group, destroy_process_group
import os
```

```
def ddp_setup():
    torch.cuda.set_device(int(os.environ["LOCAL_RANK"]))
    init_process_group(backend="nccl") # 初始化平行運算
```

```
def prepare_dataloader(dataset: Dataset, batch_size: int):
    return DataLoader(
        dataset,
        batch_size=batch_size,
        pin_memory=True,
        shuffle=False,
        sampler=DistributedSampler(dataset) # 數據平行化
    )
```

```
class Trainer:
    def __init__(
        self.gpu_id = int(os.environ["LOCAL_RANK"])
        self.model = model.to(self.gpu_id) # 將模型分配到指定 GPU
        self.model = DDP(self.model, device_ids=[self.gpu_id]) # 包裝為 DDP 模型
```

```
    def _run_epoch(self, epoch):
        b_sz = len(next(iter(self.train_data))[0])
        print(f"[GPU{self.gpu_id}] Epoch {epoch} | Batchsize: {b_sz} | Steps: {len(self.train_data)}")
        self.train_data.sampler.set_epoch(epoch) # 訓練前，進行資料洗牌，GPU會讀到不同的資料
```

```
def main(save_every: int, total_epochs: int, batch_size: int, snapshot_path: str = "snapshot.pt"):
    ddp_setup()
    dataset, model, optimizer = load_train_objs()
    train_data = prepare_dataloader(dataset, batch_size)
    trainer = Trainer(model, train_data, optimizer, save_every, snapshot_path)
    trainer.train(total_epochs)
    destroy_process_group() # 結束平行運算
```

<https://github.com/pytorch/examples/tree/main/distributed/ddp-tutorial-series>

single_gpu.py 適用一個GPU

multigpu.py 適用一機多卡

multigpu_torchrun.py 適用一機多卡

multinode.py 適用多機多卡

Execute Pytorch DDP

單節點

```
torchrun --nproc_per_node=2 YOUR_TRAINING_SCRIPT.py (--arg1 --arg2 --arg3)
```

多節點方法一：Elastic launch

```
torchrun
```

```
--nnodes=NUM_NODES 節點數量
```

```
--nproc-per-node=TRAINERS_PER_NODE 每個節點的GPU數量
```

```
--max-restarts=3 容錯3次
```

```
--rdzv-id=$JOB_ID 整數值
```

```
--rdzv-backend=c10d
```

```
--rdzv-endpoint=$HOST_NODE_ADDR:$PORT 主節點IP
```

```
YOUR_TRAINING_SCRIPT.py (--arg1 ... train script args...)
```

Slurm script for Elastic launch

```
#!/bin/bash
#SBATCH -A <ProjectID>          # iService Project id
#SBATCH -J elastic              # job name
#SBATCH -p normal               # partition dev normal normal2
#SBATCH --nodes=2               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=1     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2            # GPUS per node
#SBATCH -o %x_%j.out            # Path to the standard output file

export OMP_NUM_THREADS=1
export MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
export MASTER_PORT=$((($RANDOM % 10000 + 20000)) # random port form 20000 to 29999

SIF=/path/to/pytorch_24.05-py3.sif
SINGULARITY="singularity exec -B /work:/work --nv $SIF"

CMD="torchrun \
--nnodes $SLURM_NNODES \
--nproc_per_node=$SLURM_GPUS_ON_NODE \
--rdzv_id $RANDOM \
--rdzv_backend c10d \
--rdzv_endpoint $MASTER_ADDR MASTER_PORT \
YOUR_TRAINING_SCRIPT.py (--arg1 ... train script args...)"

if [[ "$SLURM_NNODES" = 1 ]]; then
    RUN="$SINGULARITY $CMD" # for one node
else
    RUN="srun --nodes=$SLURM_NNODES --ntasks=$SLURM_NNODES $SINGULARITY $CMD" # for multi nodes
fi

echo "$RUN"; $RUN
```

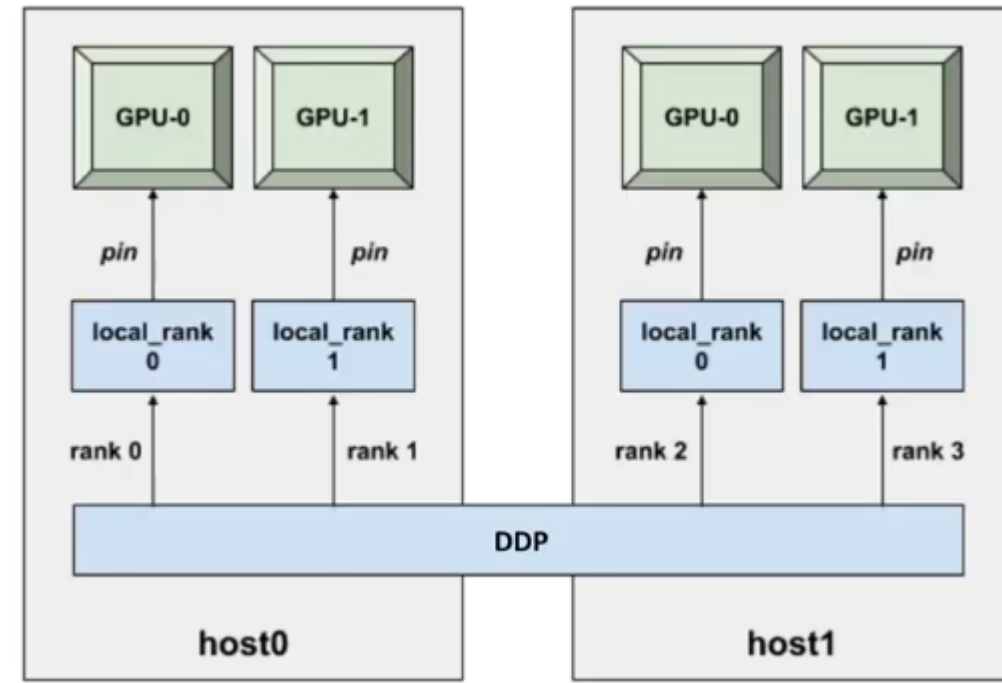
多節點方法二（多台電腦組成的叢集系統）

on Node 0

```
torchrun --nproc-per-node=NUM_GPUS_YOU_HAVE  
--nnodes=2  
--node-rank=0  
--master-addr="192.168.1.1"  
--master-port=1234  
YOUR_TRAINING_SCRIPT.py (--arg1 --arg2 --arg3)
```

on Node 1

```
torchrun --nproc-per-node=NUM_GPUS_YOU_HAVE  
--nnodes=2  
--node-rank=1  
--master-addr="192.168.1.1"  
--master-port=1234  
YOUR_TRAINING_SCRIPT.py (--arg1 --arg2 --arg3)
```



<https://www.youtube.com/watch?v=azLCUayJJoQ>

Slurm script for DDP

```
#!/bin/bash
#SBATCH -A <ProjectID>          # iService Project id
#SBATCH -J ddp                  # job name
#SBATCH -p normal               # partition dev normal normal2
#SBATCH --nodes=2               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=1     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2            # GPUS per node
#SBATCH -o %x_%j.out           # Path to the standard output file

export OMP_NUM_THREADS=1
export MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
export MASTER_PORT=$((($RANDOM % 10000 + 20000)) # random port form 20000 to 29999

SIF=/path/to/pytorch_24.05-py3.sif
export SINGULARITY="singularity exec --nv -B /work:/work $SIF"

export LAUNCHER="torchrun \
--nnodes $SLURM_NNODES \
--nproc_per_node $SLURM_GPUS_ON_NODE \
--master_addr $MASTER_ADDR \
--master_port $MASTER_PORT \
"

export TRAIN="YOUR_TRAINING_SCRIPT.py (--arg1 --arg2 --arg3)"

srun --nodes=$SLURM_NNODES --ntasks=$SLURM_NNODES \
bash -c "
  CMD=\"\$SINGULARITY \$LAUNCHER --node_rank \$SLURM_NODEID \$TRAIN\"
  echo \"\$CMD\"
  exec \$CMD
"
```


Execute Pytorch_mnist via Elastic launch

1. 下載映像檔

```
singularity pull pytorch_24.05-py3.sif docker://nvcr.io/nvidia/pytorch:24.05-py3
```

2. 編輯 slurm script

```
cd ~/nano5_tutorial/pytorch_mnist  
vi run_elastic.slurm
```

3. 執行 slurm script

```
sbatch run_elastic.slurm
```

```
#!/bin/bash  
#SBATCH -A <Project_id>          # iService Project id  
#SBATCH -J ddp                   # job name  
#SBATCH -p normal                # partition normal, normal2  
#SBATCH --nodes=2               # Maximum number of nodes to be allocated  
#SBATCH --cpus-per-task=4       # Number of cores per MPI task  
#SBATCH --ntasks-per-node=1     # Number of MPI tasks (i.e. processes)  
#SBATCH --gres=gpu:2            #  
#SBATCH -o %x_%j.out            # Path to the standard output file  
  
export OMP_NUM_THREADS=1  
export MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)  
export MASTER_PORT=$((($RANDOM % 10000 + 20000)) # random port form 20000 to 29999  
  
# get GPU count  
IFS=' ' read -ra gpu_array <<< "$SLURM_JOB_GPUS"  
GPUS_PER_NODE=${#gpu_array[@]}  
  
SIF="/path/to/pytorch_24.05-py3.sif"  
SINGULARITY="singularity exec -B /work:/work --nv $SIF"  
  
CMD="torchrun \  
--nnodes $SLURM_NNODES \  
--nproc_per_node=$GPUS_PER_NODE \  
--rdzv_id $SLURM_JOBID \  
--rdzv_backend c10d \  
--rdzv_endpoint $MASTER_ADDR:$MASTER_PORT \  
multigpu.py 10 10"  
  
if [[ "$SLURM_NNODES" = 1 ]]; then  
    RUN="$SINGULARITY torchrun --nnodes $SLURM_NNODES --nproc_per_node=$GPUS_PER_NODE multinode.py 10 10" # for one node  
else  
    RUN="srun --nodes=$SLURM_NNODES --ntasks=$SLURM_NNODES $SINGULARITY $CMD" # for multi nodes  
fi  
  
echo $RUN; $RUN
```

Execute Pytorch_mnist via DDP

1. 編輯 slurm script

vi run_ddp.slurm

2. 執行 slurm script

sbatch run_ddp.slurm

```
#!/bin/bash
#SBATCH -A <Project_id>          # iService Project id
#SBATCH -J ddp                  # job name
#SBATCH -p normal               # partition normal, normal2
#SBATCH --nodes=2               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=1     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out            # Path to the standard output file

export OMP_NUM_THREADS=1
export MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
export MASTER_PORT=$((($RANDOM % 10000 + 20000)) # random port form 20000 to 29999

SIF="/path/to/pytorch_24.05-py3.sif"
export SINGULARITY="singularity exec -B /work:/work --nv $SIF"

export LAUNCHER="torchrun \
--nnodes $SLURM_NNODES \
--nproc_per_node $SLURM_GPUS_ON_NODE \
--master_addr $MASTER_ADDR \
--master_port $MASTER_PORT \
"

export TRAIN="multinode.py 10 10" # multigpu_torchrun.py multinode.py

srun --nodes=$SLURM_NNODES --ntasks=$SLURM_NNODES \
bash -c "
  CMD="\$SINGULARITY \$LAUNCHER --node_rank \$SLURM_NODEID \$TRAIN\"
  echo \" \$CMD \"
  exec \$CMD
"
```

```
singularity exec --nv -B /work:/work /work/u8880716/sif/pytorch_24.05-py3.sif torchrun --nnodes 2 --nproc_per_node 2 --master_addr
hgpn03 --master_port 25079 --node_rank 0 multinode.py 10 10
singularity exec --nv -B /work:/work /work/u8880716/sif/pytorch_24.05-py3.sif torchrun --nnodes 2 --nproc_per_node 2 --master_addr
hgpn03 --master_port 25079 --node_rank 1 multinode.py 10 10
```

客服專線：0809-091-365

帳號與計畫諮詢信箱：iservice@niar.org.tw

技術諮詢信箱：isupport@niar.org.tw



晶創25的GPU 驅動程式與CUDA版本

```
[u8880716@cbi-lgn01 ~]$ nvidia-smi
Mon Dec 15 15:32:04 2025

+-----+-----+-----+-----+-----+-----+
| NVIDIA-SMI 550.127.08                  Driver Version: 550.127.08      CUDA Version: 12.4 |
+-----+-----+-----+-----+-----+-----+
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
| 0 NVIDIA H100 PCIe      On          | 00000000:A8:00.0 Off |                    0 |
| N/A   47C    P0              157W / 350W | 27481MiB / 81559MiB |    39%    Default |
|                                           |                      | Disabled |
+-----+-----+-----+-----+-----+-----+
| 1 NVIDIA H100 PCIe      On          | 00000000:B8:00.0 Off |                    0 |
| N/A   46C    P0              146W / 350W | 28152MiB / 81559MiB |    39%    Default |
|                                           |                      | Disabled |
+-----+-----+-----+-----+-----+-----+

+-----+-----+-----+-----+-----+-----+
| Processes: |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|      ID    ID                    |              | Usage      |
+-----+-----+-----+-----+-----+-----+
| 0     N/A   N/A       1227366      C      python                            26892MiB |
| 0     N/A   N/A       2873054      C      .../.conda/envs/kzconda/bin/python3.10 448MiB |
| 1     N/A   N/A       1227366      C      python                            28142MiB |
+-----+-----+-----+-----+-----+-----+
```

不要在登入節點執行程式!!

晶創25的Lmod軟體清單

```
[u8880716@cbi-lgn01 ~]$ module avail

----- /work/HPC_software/LMOD/public/modulefiles -----
cmake/3.24.2    hwloc/2.7.2    miniconda3/24.11.1    os    pmix/4.2.9    singularity/3.7.1

----- /work/HPC_software/LMOD/nvidia/modulefiles -----
cuda/11.6      cuda/12.4 (D)    nvhpc-hpcx-cuda12/24.7    openmpi/4.1.6 (D)    ucx/1.16.0
cuda/12.2      cuda/12.6      nvhpc/24.7                openmpi/5.0.5

----- /work/HPC_software/LMOD/intel/modulefiles -----
advisor/latest          debugger/2024.2.1      (D)    intel_ippcp_ia32/latest
advisor/2024.2          dev-utilities/latest  intel_ippcp_ia32/2021.12 (D)
ccl/latest              dev-utilities/2024.2.0 (D)    intel_ippcp_intel64/latest
ccl/2021.13.1           dnnl/latest           intel_ippcp_intel64/2021.12 (D)
compiler-intel-llvm/latest  dnnl/3.5.0           mkl/latest
compiler-intel-llvm/2024.2.1 (D)    dpct/latest           mkl/2024.2 (D)
compiler-intel-llvm32/latest  dpct/2024.2.0       mkl32/latest
compiler-intel-llvm32/2024.2.1 (D)    dpl/latest            mkl32/2024.2 (D)
compiler-rt/latest         dpl/2022.6           mpi/latest
compiler-rt/2024.2.1      ifort/latest          mpi/2021.13 (D)
compiler-rt32/latest       ifort/2024.2.1       tbb/latest
compiler-rt32/2024.2.1     ifort32/latest        tbb/2021.13 (D)
compiler/latest           ifort32/2024.2.1     tbb32/latest
compiler/2024.2.1         intel_ipp_ia32/latest  tbb32/2021.13 (D)
compiler32/latest         intel_ipp_ia32/2021.12 (D)    vtune/latest
compiler32/2024.2.1       intel_ipp_intel64/latest  vtune/2024.2 (D)
debugger/latest           intel_ipp_intel64/2021.12 (D)

Where:
D: Default Module
```

vLLM 軟體清單，包含cuda runtime

Package	Version				
aiohappyeyeballs	2.6.1			packaging	25.0
aiohttp	3.13.2	huggingface-hub	0.36.0	partial-json-parser	0.2.1.1.post6
aiosignal	1.4.0	idna	3.11	pillow	12.0.0
annotated-doc	0.0.4	interegular	0.3.3	pip	25.3
annotated-types	0.7.0	Jinja2	3.1.6	prometheus_client	0.23.1
anyio	4.11.0	jiter	0.12.0	prometheus-fastapi-instrumentator	7.1.0
astor	0.8.1	jsonschema	4.25.1	propcache	0.4.1
async-timeout	5.0.1	jsonschema-specifications	2025.9.1	protobuf	6.33.1
attrs	25.4.0	lark	1.2.2	psutil	7.1.3
blake3	1.0.8	llguidance	0.7.30	py-cpuinfo	9.0.0
cachetools	6.2.2	llvmlite	0.44.0	pybase64	1.4.2
cbor2	5.7.1	lm-format-enforcer	0.11.3	pycountry	24.6.1
certifi	2025.11.12	markdown-it-py	4.0.0	pycparser	2.23
cffi	2.0.0			pydantic	2.12.4
charset-normalizer	3.4.4	ninja	1.13.0	pydantic_core	2.41.5
click	8.2.1	numba	0.61.2	pydantic-extra-types	2.10.6
cloudpickle	3.1.2	numpy	2.2.6	Pygments	2.19.2
compressed-tensors	0.11.0	nvidia-cublas-cu12	12.8.4.1	python-dotenv	1.2.1
cupy-cuda12x	13.6.0	nvidia-cuda-cupti-cu12	12.8.90	python-json-logger	4.0.0
depyf	0.19.0	nvidia-cuda-nvrtc-cu12	12.8.93	python-multipart	0.0.20
dill	0.4.0	nvidia-cuda-runtime-cu12	12.8.90	PyYAML	6.0.3
diskcache	5.6.3	nvidia-cudnn-cu12	9.10.2.21	pymzmq	27.1.0
distro	1.9.0	nvidia-cufft-cu12	11.3.3.83	ray	2.51.1
dnspython	2.8.0	nvidia-cufile-cu12	1.13.1.3	referencing	0.37.0
einops	0.8.1	nvidia-curand-cu12	10.3.9.90	regex	2025.11.3
email-validator	2.3.0	nvidia-cusolver-cu12	11.7.3.90	requests	2.32.5
exceptiongroup	1.3.0	nvidia-cuspars-cu12	12.5.8.93	rich	14.2.0
fastapi	0.121.2	nvidia-cusparselt-cu12	0.7.1	rich-toolkit	0.15.1
fastapi-cli	0.0.16	nvidia-nccl-cu12	2.27.3	rignore	0.7.6
fastapi-cloud-cli	0.3.1	nvidia-nvjitlink-cu12	12.8.93	rpds-py	0.28.0
fastrlock	0.8.3	nvidia-nvtx-cu12	12.8.90	safetensors	0.6.2
filelock	3.20.0	openai	2.8.0	scipy	1.15.3
frozendict	2.4.7	openai-harmony	0.0.8	sentencepiece	0.2.1
frozenlist	1.8.0	opencv-python-headless	4.12.0.88	sentry-sdk	2.44.0
fsspec	2025.10.0	outlines_core	0.2.11	setproctitle	1.3.7
				setuptools	80.9.0
				shellingham	1.5.4
				sniffio	1.3.1
				soundfile	0.13.1
				soxr	1.0.0
				starlette	0.49.3
				sympy	1.14.0
				tiktoken	0.12.0
				tokenizers	0.22.1
				tomli	2.3.0
				torch	2.8.0
				torchaudio	2.8.0
				torchvision	0.23.0
				tqdm	4.67.1
				transformers	4.57.1
				triton	3.4.0
				typer	0.20.0
				typing_extensions	4.15.0
				typing-inspection	0.4.2
				urllib3	2.5.0
				uvicorn	0.38.0
				uvloop	0.22.1
				vllm	0.11.0
				watchfiles	1.1.1
				websockets	15.0.1
				wheel	0.45.1
				xformers	0.0.32.post1
				xgrammar	0.1.25
				yaml	1.22.0